



UPPSALA
UNIVERSITET

Degree Project in *****Unknown subject area*****

*****Unknown cycle*****, *****Unknown number of ***** credits

Automation of Manufacturing Optimization:

Can LLMs Help Non-Experts Solve Manufacturing Trade-offs?

**ARVID KRUSTEN
VIKTOR NORBERG**

Automation of Manufacturing Optimization:

Can LLMs Help Non-Experts Solve Manufacturing Trade-offs?

ARVID KRUSTEN

VIKTOR NORBERG

*****Master's Programme, FakeProgramme*****, XXX credits

Date: June 13, 2026

Supervisors: A. Busy Supervisor, Another Busy Supervisor, Third Busy Supervisor

Examiner: ExaminersFirstname ExaminersLastname

*****School of XXX*****

Host company: Företaget AB

Swedish title: Detta är den svenska översättningen av titeln

Swedish subtitle: Detta är den svenska översättningen av undertiteln

Abstract

Enter your abstract here!

Keywords

Artificial Intelligence (AI), Large Language Model (LLM), Discrete Event Simulation (DES), Multi Objective Optimization (MOO), Automatic Simulation Model Generation (ASMG), Manufacturing

Sammanfattning

Enter your Swedish abstract or summary here!

Nyckelord

Artificiell Intelligens (AI), Stora Språkmodeller (LLMs), Diskret Händelsestyrd Simulering (DES), Multiobjektiv Optimering (MOO), Automatisk Generering av Simuleringsmodeller (ASMG), industriell tillverkning

Acknowledgments

We express our gratitude to our subject reader Mats Gustafsson and our student colleagues for continuous feedback and rewarding discussions during the project.

We also would like to direct our gratitude to our supervisor Kaveh Amouzgar for guiding us, and for providing valuable insights and ideas. Many thanks also to Thomas Schmitt, Matías Urenda Moris and Nils Gegenmantel for helpful input and engagement in our work.

This thesis was written by the authors alone and without content generation using generative artificial intelligence (AI) tools

Uppsala, June 2026

Arvid Krusten

Viktor Norberg

Contents

1	Introduction	1
1.1	Purpose	4
1.2	Research Questions	4
1.3	Research Procedure	5
1.4	Goals	5
1.5	Delimitations	6
1.6	Structure of the Thesis	6
2	Background	8
2.1	Current Challenges in Industrial Manufacturing	8
2.2	Generative Artificial Intelligence: Core Concepts	9
2.2.1	Generative Artificial Intelligence	9
2.2.2	Large Language Models	9
2.2.3	AI Agents	10
2.2.4	Prompt Engineering	11
2.2.5	LLM Pitfalls and Hallucination	11
2.2.6	LLMs for Decision Support	13
2.3	Manufacturing Process Simulation: Application and Automation	13
2.4	Multi Objective Optimization	15
2.4.1	Evolutionary Optimization	16
2.4.2	Multi Objective Evolutionary Algorithms	19
2.4.3	Evaluation of MOO Solutions	20
2.5	Discrete Event Simulation for Multi Objective Optimization	21
2.6	Bottleneck Analysis of Production Lines	22
2.6.1	Simulation-based CONstraint REMoval (SCORE method)	22
2.7	Human-in-the-loop Simulation Technologies	23
2.8	Automation of Production Optimization: A Sociotechnical Perspective	24

2.8.1	Deskilling of the Workforce	24
2.8.2	Technology Dependence	25
2.9	Related Work	25
2.9.1	Gen-AI for Code Generation	26
2.9.2	Emerging use of LLMs for Solving MOO Problems	26
2.9.3	Automation Approaches to Simulation Modeling and Optimization	28
2.9.4	Identified Research Gap and Contributions	29
3	Methodology	32
3.1	Research Process	32
3.1.1	Research Paradigm	34
3.2	Input Data	35
3.2.1	Input Data Processing	35
3.2.2	Validity of Input Data	35
3.3	Experimental Use Case: Case A	36
3.4	Framework Evaluation and Validation	36
3.4.1	Evaluation of Usability	37
3.4.2	Bottleneck Results Evaluation	38
3.4.3	MOO Results Evaluation	38
3.4.4	General Evaluation of Framework	39
3.5	Experimental Environment	41
3.5.1	Hardware	41
3.5.2	Software	42
3.5.2.1	Python	42
3.5.2.2	Mermaid Code	42
3.5.2.3	FACTS Analyzer	43
3.6	Methodological Validity and Reliability	43
3.6.1	Validity of Method	43
3.6.2	Reliability of Method	44
4	Implementation	45
4.1	Framework Overview	45
4.1.1	LLM Agent Selection	46
4.1.2	Blueprint Models	48
4.1.3	Framework Stages	49
4.1.3.1	Input Data Analysis	51
4.1.3.2	Model Generation	51
4.1.3.3	Optimization Layer	52

4.1.3.4	Model Adaption	57
4.1.3.5	Model Evaluation	57
4.1.3.6	Model Inspection and Repair	58
4.1.3.7	Model Visualization	59
4.2	Human-in-the-loop	60
5	Results and Analysis	63
5.1	LLM API Calls	63
5.2	Optimizing Buffer Configuration of Case A Production Line	64
5.2.1	Results from the Buffer Configuration MOEA	65
5.3	Bottleneck Detection and Scenario Testing	68
5.4	Validity Analysis	71
5.5	Framework Performance over Multiple Runs	73
5.6	Analysis of the Results	78
6	Discussion	81
6.1	Technical Discussion	81
6.1.1	Insights from Development Stage	82
6.2	Sociotechnical Reflections	84
6.2.1	Keeping Humans-in-the-loop in the Advance of AI	84
6.2.2	The Cost of Technological Dependency	85
6.3	Framework Applicability in Industry	86
7	Conclusions and Future Work	88
7.1	Conclusions	88
7.2	Limitations	89
7.3	Future work	90
	References	91

List of Figures

2.1	Example of ranked bottlenecks using the Simulation-based CONstraint REMoval (SCORE)-method in FACTS Analyzer. On the x-axis is shown the improvement variables while the y-axis shows the percentage of variable improvement occurrence in solutions	23
3.1	Case A modeled in the simulation software FACTS Analyzer .	37
3.2	Exemplifying illustration of the hypervolume calculation for dual minimizing problem	40
4.1	The entire modeling and optimization process of the developed framework, from the chosen use case and data input, to the Large Language Model (LLM)-driven workflow and production changes.	47
4.2	Framework overview including the five main features: i) Input Data Analysis ii) Simulation Model Generation iii) Optimization iv) Model Adaption and v) Model Evaluation. For each step of the framework, the process is either driven by LLMs, the human operator or code execution.	50
4.3	View of the user when interacting with the framework to optimize the simulation model. In this case, the human operator chose to optimize for Work in Progress (WIP) and Throughput (TP) by finding the best buffer configuration. . . .	54
4.4	View of the user when choosing the hyper parameters of the Multi Objective Optimization (MOO) algorithm.	55
4.5	Illustration of the LLM agents and how they interact with each other. Each green square indicates that the antecedent agent has generated code which is to be executed and potentially repaired by the inspector agent.	58

4.6	An illustration of the Multi Objective Evolutionary Algorithm (MOEA) that was generated by the framework. Likewise, the mermaid code that rendered this .png image was also generated by framework. The image gives an insight of the main logic of the MOEA to the human operator.	62
5.1	A plot that was automatically generated by the framework, illustrating the MOEA results. The graph is interactive, letting the user see the Pareto front and the decision variables values when hovering over any point.	66
5.2	Resulting Key Performance Indicators (KPIs) from the original simulation model adapted with the settings from the LLM suggested points. Each adapted version corresponds to changing the buffer capacities according to Table 5.2	67
5.3	A comparison of the MOEA data generated by the framework and the FACTS Analyzer optimization for the buffer configuration problem.	68
5.4	After the initial generation of the simulation model, the user gets to choose between what type of optimization to proceed with, or to exit the framework.	69
5.5	The instruction for model adaption derived from the bottleneck SCORE method, exhibiting the top five constraints of the system.	69
5.6	The frequency analysis from the FACTS Analyzer SCORE setup. On the x-axis is shown the improvement variables while the y-axis shows the percentage of variable improvement occurrence in solutions	70
5.7	A snippet of the explanations provided by the framework of the bottleneck alleviations.	71
5.8	Evaluation of KPIs from the scenario testing of the system bottleneck (press cell 2). The scenarios 1-5 correspond to reducing the Mean Time To Repair (MTTR) by 10%, reducing process time by 10%, increasing availability by 10%, doubling capacity of antecedent buffer and doubling capacity of succeeding buffer respectively.	72
5.9	Example of reasoning error from LLM when suggesting points from Pareto front	76

5.10 Graph of the MOO results with both the framework and FACTS Analyzer solutions. The data is from run 6 of the multi run experiment in Section 5.5.	78
--	----

List of Tables

2.1	Comparison of related studies utilizing LLMs for Automatic Simulation Model Generation (ASMG), code generation & optimization and the gap that is addressed by this study.	31
3.1	Production log for Case A: Machine operational data and energy consumption	38
3.2	Hardware specifications of the experimental environment . . .	41
5.1	The Pareto front from buffer configuration problem of case A .	65
5.2	The suggested points chosen by a LLM from the Pareto front to be evaluated further	66
5.3	Comparison of KPIs of the framework and the FACTS Analyzer model	73
5.4	Comparison of the simulation KPIs from the initial and the optimized production line in ten consecutive runs of the framework. In the right most columns, the execution time and hypervolume difference to the corresponding model in FACTS Analyzer is presented.	74
5.5	Accounting of produced errors from ten consecutive runs of the framework	75

Listings

- 4.1 Code snippet out of the MOO blueprint for a buffer configuration optimization problem. 48
- 4.2 Predefined prompt to the optimizer agent for MOEA code generation. 56
- 4.3 Predefined prompt that instructs the inspector agent on how to repair the code according to the error message. 59

List of acronyms and abbreviations

AI	Artificial Intelligence
ASMG	Automatic Simulation Model Generation
CPD	Critical Process Definition
DES	Discrete Event Simulation
DT	Digital Twin
EA	Evolutionary Algorithm
EO	Evolutionary Optimization
Gen-AI	Generative Artificial Intelligence
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
MOEA	Multi Objective Evolutionary Algorithm
MOO	Multi Objective Optimization
MTTR	Mean Time To Repair
NSGA-II	Non-dominated Sorting Genetic Algorithm II
QI	Quality Indicator
SBX	Simulated Binary Crossover
SCORE	Simulation-based CONstraint REMoval
SEC	Specific Energy Consumption
TP	Throughput
UML	Unified Modeling Language
WIP	Work in Progress

Chapter 1

Introduction

Manufacturing companies have to overcome many challenges in production due to the dynamic environments and complex systems that constitute industrial production. Operating in an ecosystem of actors, manufacturing plants have to account for fluctuating prices and demands [1], while balancing internal objectives such as Throughput (TP) rate and cycle times. At the same time, many of the objectives in manufacturing are conflicting, one example being production rate and energy consumption [2], which further increases complexity in operations. Making connections between actions and consequences can be counterintuitive due to the size and complexity of modern manufacturing systems. Therefore, even domain experts may struggle to solve problems by experience or instinct. Keeping this in mind, simulation is a technology that can enable production technicians to foresee the consequences of possible actions and how they impact the performance of the system [3].

In solving manufacturing problems, simulation techniques have been established as an invaluable tool for scenario testing, process improvement, and optimization [4]. Commonly, Discrete Event Simulations (DESs) are used to model the complex interplay of different factors in real life production lines. DES is a time efficient simulation method for systems in which activities can be divided into discrete events [5]. However, experts within the field of simulation optimization often face many challenges when modeling real world systems because of their complexity [2]. In general, most systems have multiple conflicting objects, leading to trade-off solutions that require advanced optimization capabilities [2]. Developing simulation models to solve these advanced optimization problems can take many months of effort from simulation experts [4]. In practice, simulation models must be maintained and adapted to real world changes, increasing the resources needed to continuously

model manufacturing processes. Simulation techniques can be viewed as a potent tool for improving many aspects of manufacturing while also requiring time, technical skill and expert knowledge of simulation and optimization processes to use effectively. For small to medium sized enterprises, the demand of resources can make advanced simulation techniques unobtainable [4].

In recent years, with the rise in popularity of Generative Artificial Intelligence (Gen-AI), Large Language Models (LLMs) have been explored as a potential instrument to enhance simulations [6]. Gen-AI is a field within Artificial Intelligence (AI) that has gained substantial attention following the release of ChatGPT in 2022 [7]. Today, Gen-AI has many established use cases because of its ability to process natural language for text-to-text, text-to-code, or other purposes. In simulation, LLMs can assist in code generation, meaning that a DES can be modeled based on conversational input, potentially speeding up development cycles [6]. LLMs have also been shown to have the capability to improve the extraction of knowledge from simulation processes [6].

Subsequently, there has been substantial research in the field of Automatic Simulation Model Generation (ASMG), to increase the accessibility of advanced simulation modeling [4]. Elbasheer et al. [4] presents that LLMs can be of value in ASMG solutions, as they allow *non-experts* to express modeling needs in natural language to solve complex manufacturing problems of scheduling and control. In this setting, non-experts refer to people that lack experience, or training, in advanced simulation based optimization, algorithmic modeling and solving. Furthermore, Liu et al. [8] suggests that LLM technology can minimize the need for experts in optimization problems, making it more accessible. Using LLMs, non-experts can model and solve problems by conversational interactions in familiar language, reducing the need for expertise in programming and simulation optimization. This sort of technical solution has the possibility of freeing up time for experts while enabling non-experts to solve problems that were formerly conceived to be out of reach.

In manufacturing, neglect of human input in the technology paradigm of Industry 4.0 has given rise to the concept of Industry 5.0 where a more human centric approach is applied. In Industry 5.0, human-AI collaboration has become a prominent concept in academia. Industry 4.0 started as a way to address the need for higher productivity, as well as how to utilize new technology available for manufacturing systems. Generally, Industry 5.0 focuses on the use of similar technologies, but emphasizes the need to place

it in a human-centric context [9]. In other words, Industry 5.0 could be said to focus on the collaboration or symbiosis of humans and technology, rather than the standalone technology [10]. This makes it interesting to investigate the potential benefits of using LLMs to facilitate the easier and faster use of simulation technology. Simulation is stated by Xu et al. [10] to be an enabling technology of both Industry 4.0 and Industry 5.0, but studying how to make it easier to interact with could be argued to be more in line with a human-centric perspective found in Industry 5.0.

In Industry 5.0, there is an emphasis on human control over decisions in a production system. In other words, humans must understand what and how decisions are made [9]. This idea can be referred to as *human-in-the-loop*, a concept that highlights the need for human agency in collaboration with technology [11]. Although LLMs may enable easier access to simulation technology, they could be used to automate certain steps in the process without completely giving up control of decision making to an automated system. This ensures that humans have a relevant input on the optimization process while improving its availability. This could therefore be argued to be an implementation of simulation automation that is in line with the human-centric perspective of Industry 5.0. Because of this, there is a need to explore how human-machine interaction can be done efficiently and how tools like LLMs can be made to serve and cooperate with human operators [10].

To optimize production lines and overcome the challenges of complex systems, Multi Objective Optimization (MOO) stands out as an important concept in modeling real world manufacturing problems [12]. As the name implies, MOO algorithms have the ability to optimize a problem by considering multiple objectives at the same time, objectives that commonly are interrelated and mutually restricted optimization objectives [8]. The complexity arises from the interconnection of factors in production systems coupled with different and contradictory objective functions, such as production speed and energy consumption per part (Specific Energy Consumption (SEC))[8] [2]. Due to these conflicting objective functions, the solution to a MOO problem is rarely obvious. Instead, MOO algorithms produce many solutions with varying performance in the different objectives [2]. This ability to produce trade-off solutions is a more realistic representation of real world systems than assuming that an objective can be optimized in isolation without unwanted consequences to another performance indicator. A common approach to solving MOO problems is applying Evolutionary Optimization (EO) algorithms, so called Multi Objective Evolutionary Algorithm (MOEA). MOEAs is a group of heuristic based optimization

methods that solve problems with competing objectives by finding good enough solutions, while being relatively computationally effective [2] [13].

While there has been substantial research into the fields of simulation, manufacturing optimization, and Gen-AI, there have been few studies focusing on the integration of these technologies in a holistic manner. The use of AI, in the form of LLMs, can be well suited to generate, evaluate and improve the configurations of a DES manufacturing line [6]. Nonetheless, to ensure algorithmic safety and dependability, human input and feedback are crucial [9]. Therefore, there is a need for research on how manufacturing simulations can be generated, optimized and adapted, with the symbiosis of human and machine. This study intends to construct a functioning framework, that leverages LLM and DES technology, to generate and optimize a production line simulation, to advance the vision of non-expert accessible and human-centric simulation based optimization.

1.1 Purpose

The purpose of this study is to investigate how to leverage LLMs to generate and optimize simulation models in order to find good solutions to complex manufacturing problems that involves trade-offs in conflicting objectives. Within this pursuit, the project seeks to exemplify how this can be done in a human-centric approach, which allows non-expert of simulation based optimization or coding to be assisted in production optimization. Finally, the aim of this study is to investigate these questions by developing a Python based ASMG and optimization framework and evaluating its performance and usability on an example production line.

1.2 Research Questions

1. **RQ1:** How can a LLM-based, human-in-the-loop framework be used to help human operators optimize conflicting objectives in a production line?
2. **RQ2:** Can such a framework offer similar performance to traditional simulation model optimization?

1.3 Research Procedure

This study is a continuation of a previous thesis by Nils Gegenmantel [14], in which the author built a LLM driven framework for generating and adapting simulations of a manufacturing line. In the previous study, optimization of the simulation model was performed mainly by a utilization bottleneck analysis, which led to some parameter changes of the simulation model. To further increase the usability of this framework in a manufacturing setting, as a tool to help non-experts with MOO, this study intended to expand the framework, by modifying the optimization layer, to enable LLM agent assisted MOO.

To achieve this, the framework was further developed with additional LLM agents that were tasked with assisting in optimizing multiple objectives of a production line, for example TP and energy consumption, by changing parameters such as buffer sizes or process times. For this purpose, OpenAI's LLM models [15] were used and implemented as agents in Python. The production line was first implemented in FACTS Analyzer, a DES software. By having the production line modeled in FACTS Analyzer, it could be used as a benchmark to see how the simulation generated by the framework performed. To thoroughly test the frameworks performance, quantitative tests were applied to measure the frameworks performance in comparison with traditional implementations of MOEAs. Furthermore, the usability of the framework was tested by evaluating attributes and errors that frequently occurred during test runs.

1.4 Goals

To fulfill the purpose of this study and gain valuable insight on how LLMs can be used as a tool to allow non-experts to find trade-off solutions to production problems, this study has multiple goals as presented in the following list.

1. Develop the framework on automatic simulation generation originally created by Gegenmantel [14], by leveraging LLMs as a means to allow non-experts to optimize multiple objects of a production line.
2. Evaluate the usability of the extended framework and make a comparison to the more traditional approach of using a simulation engine like FACTS Analyzer.

By encompassing these goals, the addition of the ability to perform MOO will expand the usability of the framework, as few real world manufacturing

problems consider a single objective. Another expected contribution is insights on if the combination of technologies such as LLMs, DES and MOEA can give more actors access to the benefits provided by simulation based optimization. By understanding how this can be done and the usability of the framework, there is the possibility of increased optimization ability in industry. Lastly, the study will also provide an example of how an implementation of a LLM driven framework for simulation based MOO can be done. This could then serve as a model for future implementation and research.

1.5 Delimitations

This study does not intend to compare how different LLM models perform in solving MOO problems in the context of the framework. Although, different OpenAI LLM models are applied for different tasks in the framework, no comparison will be made of LLM models from other companies or organizations than OpenAI. Therefore, the results yielded by the OpenAI agents in this study will be generalized to LLMs in general. Thereby treating the LLM models used as a specific case that is generalizable to other LLMs.

Furthermore, any hyper parameters used for the LLM calls are derived by previous research, and are not systematically tested. Therefore, this study does not intend to propose any optimal hyper parameters for the LLMs in the developed framework.

The simulation capabilities of the framework, including agent prompts, simulation model blueprint, and general logic, have been kept largely intact according to the work of [14] and [16]. Therefore, the generated simulation models are only used as the base of the developed MOO framework, without further attempt to improve the simulation generation part of the framework.

1.6 Structure of the Thesis

The remainder of the thesis is structured as follows. In Chapter 2, a background is given to the topics of the project, including among others, LLMs, MOO and simulation technologies. Related work is then presented of relevant previous research on the issue. In Chapter 3, the methodology is outlined, starting from the research process and also covering the data and experimental settings, framework evaluation and a discussion of validity and reliability. In Chapter 4, the technical implementation is covered, with a careful account of the developed framework and its logic. In Chapter 5, the main results of the

study are declared, with an analysis in the context of the research questions. In Chapter 6, the results are discussed in terms of the technical framework, along with a piece on the sociotechnical aspects of the project. Finally in Chapter 7, the thesis comes to a conclusion, with a section on possible avenues for future work.

Chapter 2

Background

This chapter aims to establish the theoretical foundation upon which this thesis is based. The main goal of this chapter is two-fold. Firstly, this chapter intends to explore and illustrate the main theoretical and technical concepts of this study. Since this thesis is positioned within the field of Industrial Engineering and Management, technical aspects are therefore comprehensively explained to take into account the diverse academic backgrounds common within the field. Secondly, in Section 2.9, the latest research within the topics of this thesis are reviewed to give the reader a better perception on what has been previously done, and the research gap this thesis means to contribute to.

2.1 Current Challenges in Industrial Manufacturing

The modern manufacturing industry is an important part of the economic growth that the world has experienced in the last century [17]. Much of the industry has achieved a high standard of production and efficiency. Still, there are future challenges facing the industry. There is an increasing focus on environmental sustainability that is putting pressure on manufacturing organizations to achieve higher efficiency with less environmental and ecological impact [18].

Efforts of optimizing production lines using data driven and AI approaches are anticipated to solve or mitigate some parts of these challenges [17]. However, there are also problems associated with this. One of these is that the high standard that current manufacturing process have require new tools and automation approaches to live up to very stringent requirements [17]. They

also need to be able to handle already current challenges such as dynamic conditions, high complexity and fast development cycles in order to be useful [18]. Still, the data-driven approaches are presented in the theory of industry 4.0 and 5.0 as having a large scope for future production efficiency gains [10].

2.2 Generative Artificial Intelligence: Core Concepts

In this section, Gen-AI will be presented to give the reader an understanding of its main features. Other technologies related to Gen-AI, such as LLMs and AI Agents, that are used in the study are then presented.

2.2.1 Generative Artificial Intelligence

Generative AI, or Gen-AI, can be thought of as a subgroup of AI systems which have the capability to generate material on their own [19]. This material can be of differing types of data like text, images, video and many others depending on the model that is used [19]. These models are trained on datasets of the different types of data in order to understand patterns and structures, which is then used to be able to generate new and unique data with similar characteristics to the training data [19].

2.2.2 Large Language Models

LLMs are a type of Gen-AI that is especially good at working with natural language [20]. Modern LLMs are capable of achieving a level of performance otherwise mostly seen in humans for various cognitive tasks [20]. The main reason for their performance with natural language is their ability to generalize their training to multiple tasks, enabling them to complete tasks and interact in ways that are not included in their training data [20]. The reason for the ability to move beyond trained scenarios is the scale of the trained models [20]. The drawbacks related to the size of these models is that the training becomes slow and that advanced hardware is required, leading to a higher cost of running the models [20].

To process natural language, the LLM first parses a text and divides it into units called *tokens*, which can be words, symbols or characters [20]. The LLM then uses different types of algorithms in order to determine which tokens are

considered the most important [21]. The LLM can based on this then predict a sequence of words that it decides is reasonable based on the input tokens.

Temperature is a parameter setting that is relevant to the functionality of LLMs. The temperature determines how the model regulates the stochasticity of its outputs [22]. This can be said to control the amount of risk the model is willing to take when generating a sequence of words. A lower setting for the temperature will lead the model to make predictable choices, while a higher setting will increase the variability of answers by increasing the stochasticity [22]. In short, the model will calculate probabilities for possible next words, the temperature parameter setting will then influence to which degree the model follows these probabilities. With a lower setting corresponding to the model following the probabilities more closely. According to [23], a lower temperature setting tend to be beneficial for code generation. Specifically, a temperature lower than 0.5 yield the best performance, since the LLM becomes more deterministic [23].

2.2.3 AI Agents

AI agents are an application that utilizes AI in order to achieve a task by reasoning about its given goal and context, as well as being able to use different tools [24]. The centralized decision maker for the agent is the model. The model most often refers to a LLM that can interpret instructions for the agent [24], in which case they are also known as LLM agents. This can be interpreted as the LLM being the brain of the agent which expands the potential use of the agent, in part through the multimodal perception that LLMs offers [25]. One advantage of using LLMs with agents is the fact that LLMs have gone through extensive pre-training and require little to no specific training for many tasks that these agents are asked to perform [25]. Another advantage is that LLM agents can interact effectively with each other which is enabled by LLMs natural language comprehension and generation. This supports the ability to get agents to collaborate or compete [25]. This ability allows them to be flexible, and means that agents can communicate with humans, different types of standardized systems, as well as other agents [25].

LLM agents have another benefit in many applications, which is their memory. Agents are able to access earlier interactions when necessary to solve problems and develop strategies [25]. However, something to consider with the memory is that LLM agents process these prior interactions as natural language and if these interactions become too large it might go beyond the capability to handle them and lead to some memory loss [25]. For agents the

memory contains its *context window*, this is the information that the LLM has access to, like the prompt and prior interactions. [20]. The prompt can involve instructions or examples which provide guidance for tasks and strategies [20].

Nonetheless, LLM agents have some drawbacks, they do not memorize training data but rather generalize from it, their knowledge may lack in specific areas and they lack transparency, which can be a problem if hallucination (see Section 2.2.5) occur [25]. Tool usage is another way to expand the capabilities of LLM agents, that can mitigate some of the drawbacks [25]. LLM agents are well suited for tool integration and tools that are designed to accomplish a specific task can result in more robust and transparent outcomes from the agents [25]. By using prompts to describe the function of a tool and how to use it, agents can often effectively learn to use them reliably [25]. The term tools can include many things but some examples could be to use specific code libraries to accomplish tasks [25].

2.2.4 Prompt Engineering

Interaction with LLMs is done through *prompting*, to which the trained LLM generates responses. The structure and syntax of the prompt can have an impact on LLMs functionality in ways that are not apparent to humans [20]. *Prompt engineering* is the term given to the efforts to get a LLM to interpret the prompt in the desired way by designing them in specific ways. There exist a few different common ways for structuring and using prompts. *Zero-shot prompting* refers to LLMs capability to generate answers without the prompt containing any examples for how the answer should look or be structured and is therefore relying on the models pre-existing training [26]. *One-shot prompting* differs from zero-shot prompting by providing an example for the task that the LLM is asked to perform, meaning that the model does not only have to rely solely on prior training [26], which can improve the reliability of output. *Few- and many-shot prompting* then is when multiple examples are provided of the task to the LLM.

2.2.5 LLM Pitfalls and Hallucination

As mentioned above, LLMs have a remarkable ability to generalize from their training data in order to answer questions or generate material that is not specifically present in the training data. This is one of their strengths but can also cause problems. An example of this is that when the LLM does not have a direct answer to draw on from the training data, it is possible that the model

will produce answers that sound plausible but are actually incorrect, which is called *hallucinations* [27]. This can happen because of different strategies that LLMs use, like contextual learning or probabilistic language modeling [27]. Contextual learning means that the LLM uses pattern recognition to generate plausible answers while probabilistic language modeling means that the LLM simply predicts the most likely sequence of words, these strategies mean that there is no grantee that the output is factually correct [27]. Hallucinations are therefore a limitation that users of LLMs need to be aware of in order to minimize the potential problems that can arise from the phenomenon.

Another problem that has arisen when Gen-AI and LLMs gained a large user base is the over reliance and trust that users place in the answers they receive [28]. This can especially be a problem when the user is not well informed about the topic that the LLM is prompted in, where an expert in the area would be more able to recognize faults [28].

Following on the sometimes too large trust that is placed in outputs from LLMs is the lack of understanding of how models work that can lead to problems of trust. A way that this can manifest is when looking at the size of the context window of LLM models and how they are not always able to encompass the entire input [28]. If this is not understood by the user, it may lead to them thinking that the answers produced by a LLM might have incorporated all information that is present in the input, but that is in fact not utilized as the context window could not contain its entirety.

There is also the problem of LLMs lack of transparency for evaluating answers. LLMs often comes across as a so called "black box" solution, referring to the fact that the user has no intuition or information on how the model arrives at the answer based on the provided prompt [28]. This decreases the ability of users to verify answers that are given by a LLM.

Another thing that needs to be considered is the ability of LLMs to generate code in different programming languages, as well as how it depends on which languages are used for the natural language prompts. A problem can be seen in that many LLMs exhibit better performance when prompted in english compared to other languages [29]. Looking at which programming language is used a study by Wang et al. [29] found that Python is a language that LLMs seem to perform well for code generation. However, the study also finds that there exist strategies to mitigate the performance difference on both natural- and programming languages by modifying the prompts used as well as instruction tuning, where the LLM is provided with high quality instruction-answer pairs as context [29].

2.2.6 LLMs for Decision Support

One domain in which LLMs have gained traction is within *decision making under uncertainty*, i.e. making decisions from a set of possible choices, based on some preset goals and uncertainties in the world [30]. However, directly prompting a LLM for decision making can yield poor results, especially when problem complexity increases [30]. Namely, it tends to produce non-optimal decisions because of multiple reasons: fixation on prompt specific formulations while ignoring broader context, tendency to be overconfident with lack of calibration, and trouble to align with users intent and utilities [30]. On top of that, due to the nature of LLMs, direct prompting leads to decision making that is difficult to unravel and understand for the user.

2.3 Manufacturing Process Simulation: Application and Automation

Organizations that engage in manufacturing of goods are often made up of complex systems. How the parts of the systems interact with each other and the external environment can be hard to pin down when forces such as competition and technological practices change [31]. This has led to organizations utilizing different tools, such as simulations, to understand the current state of their processes, planing for the future and provide decision support for users [31]. Because of this, manufacturing process simulations have been a part of manufacturing planning and development for some time as a way to model a simplified version of the real system.

One of the biggest advantages with using simulations in this way is that it grants the ability to experiment with the system without the need to change the real world production line [31]. This can be preferable as experimentation with the real system can be expensive, require a lot of time and in some cases be impossible [31]. This could be due to safety considerations, the risk of damage to the system, or the need to halt production while it takes place.

A simulation can be described as a simplified model of the real world system that is "run" digitally through simulated time in order to provide a representation of how said real world system would perform in the same situation [31]. There are a lot of different types of simulations that can be used for different setting, one commonly used for manufacturing systems is DES [31]. DES is a type of simulation that involves a system with individual units and discrete events that affects these units. There are many potential

applications for DES, some of the more common according to Skapinyecz [5] are the following:

- Modeling and optimization of production logistics systems
- Production scheduling and control
- Modeling and optimization of supply chains
- Modeling and optimization of various types of material flow systems

This wide application field and popularity is possible due to DES being flexible as well as the wide array of DES software available, making it an relatively easy method to use [5]. These software commonly provide a graphical interface that makes DES accessible to many and include the ability to create model behavior that is enough for most applications, they can also have added flexibility by allowing users to extend the model with custom code for further functions [5]. A DES of a real world manufacturing system can be considered to be a Digital Twin (DT), i.e. a dynamic and virtual representation of the physical system. Partly because of this, DES has risen in use lately, as there is a need for decision making capabilities utilizing DTs [5].

In manufacturing, the application of AI technologies can transform traditional simulation methods by driving more agile and efficient systems [6]. Notably, AI models can be useful for enhancing simulations in a DT framework by allowing autonomous adaptations in real time and providing fast and accurate predictive analytics and scenario testing [6]. This is highly relevant since simulation models serve as a valuable tool for decision makers to do optimization, scenario testing, and predictive analysis [4]. However, as Elbasheer et al. [4] highlights, simulation processes are time consuming and require domain knowledge. Furthermore, with rapid change and improvement of the production environment, the simulations need to be continuously adapted and maintained. To bridge this gap of accessibility to advanced simulations, there has been substantial research into the field of ASMG, in which AI technologies have played an eminent role [4]. ASMG refers to the fully or partially automatic generation of a simulation model development process, and the real time synchronization of such a model [4]. Within the topic of ASMG, LLMs have emerged as a possible paradigmatic shift to address the limitation of the necessity of expertise in building simulation models [4]. This because of LLMs ability to generate code based on textual inputs which can be applied to generate simulation models.

2.4 Multi Objective Optimization

Multi objective optimization refers to optimization problems for which two objectives functions are to be either minimized or maximized [2]. In this report, MOO hereby only refers to optimization of exactly two objective functions. The goal of MOO is to find the most attractive objective values by changing some *decision variables*. How the decision variables affect the objective variables is determined by some objective function $f_m(\mathbf{x})$, where a solution $\mathbf{x} \in \mathbf{R}^n$ is a vector of n decision variables: $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$.

A core concept in MOO is how optimal solutions are defined, since usually no one solution is better for both objectives. This means that MOO algorithms find trade-off solutions where an improvement in one objective leads to the deterioration of the other. The term *domination* is used in this context and is defined by [2] (p.9) according to:

Definition 1 A solution $\mathbf{x}^{(1)}$, where a solution $\mathbf{x} \in \mathbf{R}^n$ is a vector of n decision variables, is said to dominate the other solution $\mathbf{x}^{(2)}$, if both the following conditions are true:

1. The solution $\mathbf{x}^{(1)}$ is no worse than $\mathbf{x}^{(2)}$ in all objectives. Thus, the solutions are compared based on their objective function values
2. The solution $\mathbf{x}^{(1)}$ is strictly better than $\mathbf{x}^{(2)}$ in at least one objective.

In other words, a solution $\mathbf{x}^{(1)}$ dominates the solution $\mathbf{x}^{(2)}$ if, and only if, $\mathbf{x}^{(1)}$ is better than $\mathbf{x}^{(2)}$ in one object and not worse in both objectives. By this definition, *non-dominated* solutions are better than other solutions in at least one objective and no other solution is better in both objectives [8].

In MOO problems, all generated solution points that are not dominated form a set of non-dominated solutions. One property of such a set, given any two points in the set, is that a gain in one objective from one point to the other lead to a sacrifice in the other objective [2]. This property is usually referred to as the *trade-off* characteristic of MOO problem. By this property, MOO algorithms seek to find many non-dominated points in the objective space, which make up a front of so called *Pareto-optimal solutions* [2]. Since no point on the Pareto front is better in both objectives, it is typically up to the practitioner to choose between the non-dominated solutions depending on higher level information like experience or goals.

For these reasons, MOO problems forces the user to choose a solution with acceptable performance in both objectives, since a gain in one objective means

a trade-off or a sacrifice of the other objective. In this context, there are two main goals of multi objective optimization, which [2] describes as:

1. Find a set of solutions which lie on the Pareto-optimal front, and
2. Find a set of solutions which are diverse enough to represent the entire range of the Pareto-optimal front.

At this point, the preferred solution has to be chosen by the user, which can prove to be a difficult task, since multiple solutions are mathematically optimal [2]. One common approach to choosing a preferred solution is to look at the "knee-point" region of the Pareto front. A knee-point is often considered to be the best compromise, since in this region, a small improvement of one objective leads to a disproportionately large degradation in the other objective. Visually, in a two dimensional objective space, the knee-point region is located where the Pareto front "bends" the most, resembling a knee joint.

There are two main kinds of MOO decision philosophies: *Preference based* (a priori) and *Ideal Based* (a posteriori). Preference based methods let the decision maker input information on which objective is deemed more important in the optimization process [32]. This is typically implemented by a weight vector which gives the objectives relative importance [32]. The weights can be used to turn the MOO into single objective functions using an aggregator function, like the weighted sum [32]. Generally, the preference approach limits the solution space of the algorithm according to the decision makers input [32]. In the ideal based approach, the algorithm first generates an exhaustive Pareto front of trade off solutions, from which the decision maker chooses the preferred solution [33]. Therefore, the key difference between these two approaches is when the decision maker is involved. In the preference based methods, the practitioner have to decide between objectives *a priori*. In contrast, the ideal based approach lets the decision maker be involved *a posteriori*.

2.4.1 Evolutionary Optimization

EO algorithms, or Evolutionary Algorithms (EAs), copy the theory of evolution from biology by implementing a population based approach in which each iteration has more than one solution that evolves into a new population of solutions in the next iteration [2]. In each iteration, the population of solutions is derived from the best parent solutions in the previous generation, in a stochastic process. Ideally, as the algorithm progresses, the solutions are

continuously improved by iteratively emphasizing better population members [2]. EAs are a common ideal based approach for solving MOO problem, as they are suitable for generating a diversified Pareto optimal front in a single run.

EAs are widely used in multiple fields of academia and industry due to its advantageous properties. Firstly, EAs do not require any gradient or derivative information in its search process [2]. This means that EO algorithms are direct search procedures, making them applicable for a range of optimization problems, especially in complex systems where such information is not available [2]. Secondly, EAs are comparatively easy to implement compared to other optimization algorithms. Thirdly, EAs are flexible and can be useful in many applications since the algorithm can be adapted to problem specific challenges [2].

Furthermore, the population based approach of EAs provide additional advantages. That is, it can enable the algorithm parallel processing power, leading to a relatively computationally quick search procedure. Moreover, EAs are an attractive choice for MOO problems since the population allows EAs to find multiple optimal solutions in a single simulation run, which becomes possible solutions in a trade-off among multiple objectives [2]. Lastly, EA procedures are stochastic, resulting in superior global search, as they can search wide in the solution space.

Typically, EAs iterates between populations by four main operators: (i) Selection, (ii) Crossover, (iii) Mutation and (iv) Elite-preservation. Though, many EAs are commonly initialized by a randomly created set of solutions which becomes the first population (e.g. $\mathbf{x} \sim \mathcal{U}(a, b)$). Preferably, meta knowledge of the problem can be of use when initializing the first population, since this can lead to faster convergence [2]. By randomly initializing the first population, EO algorithms typically exhibits a more global search in early iterations, to find promising local regions to which it converges.

In the (i) selection operator, the population solutions are evaluated by some well defined quantitative measure. Commonly, the selection operator choose better performing solution with higher probability, solutions that end up in a *mating pool*, i.e. solution that are later used in step (ii) and (iii) [2]. Notably, many variations of the selection operator exist. One being the *tournament* selection, where pairs from the population are picked randomly and the best performing solution advances to the mating pool [2].

Both the (ii) crossover and the (iii) mutation operators are concerned with producing a modified child population. Firstly, the crossover operator randomly chooses two or more parent solutions from the mating pool and

generates one or more child solutions by interchanging information (i.e. parameter values) between the parent solutions [2]. Generally, the crossover operator is applied with a probability of $p_c \in [0, 1]$, corresponding to the proportion of population members in the mating pool that participate in the crossover operation [2]. The remaining solutions are simply copied to the child population. Typically, a distribution that depends on the difference between the parent variables is defined to generate the variables of the child solution [2]. However, the specific details of the crossover operator can vary. The crossover operator allows the algorithm to leverage the current best solutions in order to possibly generate better ones.

A specific crossover operation that can be used is the Simulated Binary Crossover (SBX) operator. Many applications using binary-coded EAs use a single crossover point, where two parent strings switch bits on one side of the crossover point and create two child strings [?]. This can create problems with producing valid child strings when using real valued variables [?]. In order to simulate a single point binary crossover on real valued variables a probability distribution is therefore used to create the child strings, which mitigates these problems [?]. The SBX operator is one of the most commonly used crossover operators in EO [34]. The SBX operator uses a probability parameter like many other crossover operators, but also a parameter η controlling the diversity of the child variable values compared to the parents values [35]. The higher the value of η the closer the child values resemble the parent values.

The (iii) mutation operator is then applied to the child solutions which were created by the crossover operator. In similarity with gene mutation of biology, each variable of the child solution is mutated by a probability of p_m , usually set to $1/n$, where n is the number of decision variables [2]. This ensures that, on average, one variable is mutated in each iteration of the algorithm. One common approach, for continuous variables, is to let the mutated variable be distributed by a Gaussian distribution with a predefined variance and a mean equal to the previous value of the variable [2]. The mutation operator allows the algorithm to explore a local solution space independently of other population members.

Another commonly used mutation operator is the Polynomial Mutation (PM) operator [36]. The PM operator uses a mutation probability parameter deciding if a variable should be altered and a mutation distribution index η_m , which decides the extent to which the mutated variable value is changed [36]. The mutated value is achieved through a polynomial probability distribution centered on the original variable value while the η_m value decides the distribution [36]. Larger η_m values tend to produce point closer to the original

value while smaller values produce points further from the original point [36].

Finally, the (iv) elite-preservation operator combines the old parent population with the newly generated child population and chooses the best performing solutions from both generations to finalize the next generation. This operator ensures that the algorithm has a non-degrading performance [2].

Lastly, the algorithm needs to come to an end by some termination criteria. EAs are heuristic based, meaning they are designed to find good enough solution within a reasonable time frame. Therefore, a common termination criteria is a predetermined number of generation in which the algorithm is supposed to run. In problems when there is a clear goal, the EA can terminate when fulfilling a target goal or solution is found. EAs can also be terminated based on their convergence rate. Being heuristic based, EA do not exhaustively search the solution space, which means that they are not guaranteed to find globally or even locally optimal solutions.

The flexibility of the operators and the possibility to alter details in the general EA procedures makes them able to solve many real world problems. Particularly, EAs can handle mixed variable types, as they can be employed on both combinatorial and continuous variable problems [13]. EAs can optimize non-differentiable objective and constraints, non linear problems, multiple optima and large search spaces.

2.4.2 Multi Objective Evolutionary Algorithms

As briefly touched upon in Chapter 1, multi objective evolutionary algorithms is a group of heuristic based optimization methods for solving problems with multiple, and generally opposing, objectives.

EO is a common methodology for solving MOO problems, and the group of algorithms that intend to solve MOO problems by EAs is hereby referred to as MOEAs. Notably, due to the heuristic nature of EAs, such methods do not guarantee finding the true Pareto-optimal points. However, MOEAs have been shown to be successful in a wide range of MOO tasks in finding good enough solutions, in a computationally effective manner [13] [2].

In the literature, three main groups of MOEAs have been developed and extensively used: non-dominated sorting based, indicator based and reference based methods [32]. Non-dominated sorting based methods compare population solutions to each other to determine which ones are non-dominated, emphasizing classes of non-dominated solutions while preserving diversity on the objective space [2]. One commonly used non-dominated sorting based algorithm is the Non-dominated Sorting Genetic Algorithm II (NSGA-

II), which is a widely used baseline for comparing algorithmic performance [36]. Indicator based methods replace the Pareto dominance as the selection criterion with a single indicator, like hypervolume (see Section 2.4.3) [32]. Reference based methods break down the problem into single objective subproblems, according to references such as weight vectors, reference point or reference vectors, and execute many independent single objective optimizations [2]. A commonly used subset of reference based methods are decomposition based methods [13]. Generally, decomposition based methods are considered more scalable and have a better convergence rate, compared with non-dominated sorting methods, with increasing population size and objective numbers [37]. This is mainly due to the fact that the domination comparison becomes less effective with many objectives, since the ratio of non-dominated solutions typically increases with the number of objectives [32]. However, as stated by the "No Free Lunch" theorem, no single optimization algorithm can consistently outperform others in all problem classes [38]. In other words, the best performing optimization algorithm will always be problem specific.

2.4.3 Evaluation of MOO Solutions

MOO problems are not as straight forward to evaluate as single objective optimization. For a single objective optimization different optimizers can be compared by which produces the best solution. For MOO it is different as a set of Pareto optimal solutions must be compared against each other. One of the most used way to evaluate MOO solutions is with visual inspection of the plotted solutions as this gives insight into the shape of the Pareto front and the evaluated solution space [39]. Quality Indicators (QIs) offer a way to quantitatively compare different sets of solutions [39]. There are many different QIs and they can be used to compare different optimizers in order to find the most promising and best performing [39]. The quality of a MOO solution set is usually decided by how well it approximates the Pareto front. This can be said to be approximated by four aspects, namely:

- **Convergence:** How close the set of solutions are to the Pareto front [39].
- **Spread:** The region of the solution space that the set covers [39].
- **Uniformity:** Considers how even the solution distribution of the set is, similar spacing among solutions is desired [39].

- **Cardinality:** Refers to the amount of solutions in the set [39].

One QI that is widely used is the hypervolume measure, which calculates the volume of the area enclosed by the set that is evaluated and some reference point decided by the user [39]. Hypervolume gives a comprehensive look at the performance of an optimizer as it, at least to some degree, can be said to indicate the performance on all four aspects mentioned above [39]. It is also one of the most used QIs partly because of the ease of using it, which stem from it not needing a reference set of the real Pareto front, making it easier to use in real world problems. [39]. An issue with using the hypervolume is the complexity to calculate it, which is exponentially increasing with regards to the number of objectives. Another issue is to how the reference point should be chosen. There is no definitive consensus but common practices include using the nadir point of the Pareto front or 1.1 times the nadir point of the collection of compared sets [39]. The nadir point, or vector, is defined by Deb et al. [40], p. 571, as "An objective vector $z^{nad} = (z_1^{nad}, \dots, z_M^{nad})^T$ constructed using the worst values of objective functions in the complete Pareto-optimal front P^* is called a nadir objective vector"

2.5 Discrete Event Simulation for Multi Objective Optimization

The use of DES and MOO has for a long time been utilized for evaluating industrial production systems. DES are particularly well-suited for modeling the complex interplay of the different factors at play in a production system [3]. They are suited for complex models as simulation based optimization does not require a mathematically well-defined objective function. Furthermore, MOO is used as it gives the ability to optimize a problem by considering multiple objectives at the same time. This can be hard as these objectives often can be interrelated and mutually restricted optimization objectives [8]. An example of such objectives could be TP and SEC [3]. The goal is to consider these different objectives and find different Pareto optimal solutions [8]. One of the most popular ways of doing this is by utilizing evolutionary algorithms [8]. A particular positive feature of pairing simulation, like DES, and MOEAs depends on the fact that by design MOEAs use population for each iteration, which means that in a single simulation run many non-dominated solutions can be found [2]. As the simulation and MOEA combination produces multiple Pareto optimal solutions there is still need for further analysis in order to determine the preferred solution, as none are objectively better than all others

[2]. This need for further analysis of the result from DES and MOEA could be suitable for some kind of AI implementation.

2.6 Bottleneck Analysis of Production Lines

Bottlenecks in a production line refers to when an operation or other part of the production line is the limiting factor of the TP that the line can achieve. The elimination or reduction of a bottleneck is therefore a way to increase the efficiency of the entire line [41]. Because of this it is of high interest to be able to find the bottleneck of a production line.

2.6.1 Simulation-based COnstraint REmoval (SCORE method)

The Simulation-based COnstraint REmoval (SCORE) method is a sophisticated method for bottleneck detection using simulation based optimization to find and rank multiple bottlenecks and their causes [42]. The SCORE method is carried out by formulating a MOO problem where TP constraints to the system are used as input variables [42]. The constraints can, for example, be the availability, Mean Time To Repair (MTTR) and process time of all the machines. These constraints are then gradually relaxed (changed) during the MOO process [42]. The optimization usually optimizes for maximizing TP and minimizing *Improvements*, meaning that the MOO tries to find the best performing system according to TP while the constraints of the system are kept as close to the original value as possible [42]. By doing this, the idea is that the biggest improvement in TP will be seen by removing the most significant constraint(s) [42].

A frequency analysis can then be used in order to rank the bottlenecks [42]. This is done by analyzing the frequency of which variable changes are present in the considered *best solutions* of the MOO problem [42]. What is considered *best solutions* are not absolute, but can for example be the solutions of rank 1-5 [42]. By doing the frequency analysis the ranking is both by operation and type of change, as can be seen in Figure 2.1.

The SCORE methods result can be of great help in optimization as multiple bottlenecks can be detected and ranked [42]. However, as it uses the same type of algorithms as MOO the complexity of the problem grows fast as many input variables are considered, which can lead to the SCORE method requiring long runtimes [42].

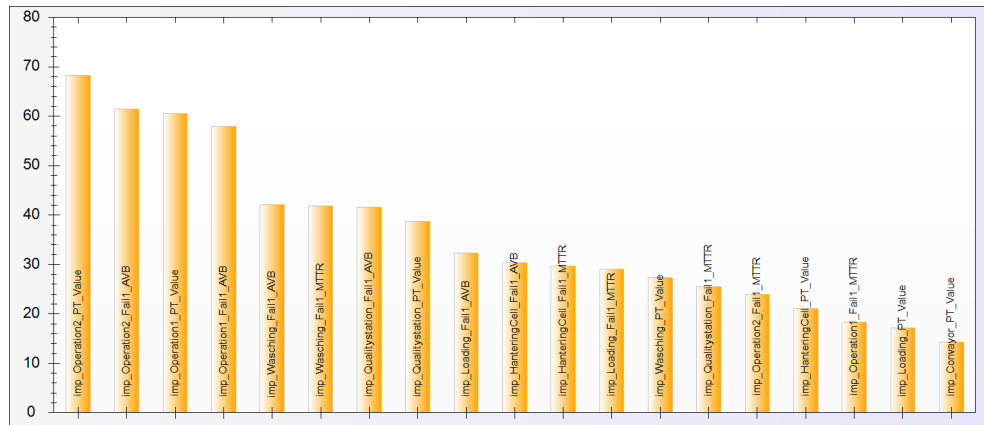


Figure 2.1: Example of ranked bottlenecks using the SCORE-method in FACTS Analyzer. On the x-axis is shown the improvement variables while the y-axis shows the percentage of variable improvement occurrence in solutions

2.7 Human-in-the-loop Simulation Technologies

During the last ten years the manufacturing industry has had a focus on digitalization and automation that has been summarized as the paradigm of Industry 4.0. One example is early research on automated optimization, which sought to mitigate reliance on human experience by employing evolutionary algorithms [38]. In recent years, this technology paradigm has been supplemented by the ideas of Industry 5.0, which uses mostly the same technologies but tries to apply them with a human-centric perspective [9]. Looking at what Xu et al. [10] calls enabling technologies for Industry 5.0 we find individualized human-machine interaction technologies, simulations as well as AI to detect casualties in complex systems. Following these enabling technologies for Industry 5.0 point to their importance in today's and tomorrow's production systems.

When working with AI and simulation for decision support there arises possibilities for collaboration with humans [11]. The goal of total automation associated with Industry 4.0 could be altered for a more human-in-the-loop approach. An automated decision by AI could instead be made to resemble a stage-gate analysis where a human operator is involved in the different steps, both to validate the decisions but also to be informed of its reasoning [11]. The visualization of data from the production system as a part in this decision process is an important part of the decision process [11], which might be

achievable in part with simulations. So by involving human operators more, with information flows and validation of automated processes, the process might be more compatible with the Industry 5.0 paradigm.

2.8 Automation of Production Optimization: A Sociotechnical Perspective

In this section some sociotechnical aspects of automation of production optimization is investigated and how such a task can broadly affect the organizations that employ such systems.

2.8.1 Deskillling of the Workforce

There are potential negative aspects with automating processes by using AI technologies. When tasks are automated by AI, it can result in humans losing the skills necessary to perform those tasks [43]. This is called *deskilling* and usually occurs when technology becomes more advanced and replaces humans in a particular task, resulting in humans exercising that skill less [43]. This can result in skill depletion in organizations where tasks are automated by AI, which can be difficult to reverse [43]. A particular problem, pointed out by Krook [43], can happen when decision-making skills are outsourced to AI. The problem with decision making being automated by AI is that humans lose influence over the actions that are taken [43]. In some cases, as AI becomes more advanced, it might be seen as delivering ground truth answers that can not be challenged by humans [43]. This will further undermine the decision-making skills of an organization. There could be ways to limit these effects by incorporating a more collaborative human-machine interactive approach to AI, instead of a more clear automation approach [43].

Another, more long-term problem that arises from automation of cognitive work by AI is the loss of learning opportunities for entry level workers [44]. When the routine and less complex tasks of a process can be automated, experts in the field are still in demand, but the tasks that are being automated are often the ones that entry level workers perform [44]. The automation of these tasks therefore impact the learning opportunities that are necessary to create future experts [44]. This could lead to problems in the future where less experienced or expert workers are available.

2.8.2 Technology Dependence

A potential problem with utilizing AI, and specifically LLMs, for automation approaches is the technological dependence that the organization might be locked in to. This because of the very concentrated market of the LLM selections [45]. The development of LLMs require a vast amount of resources in technical knowledge, data accessibility and computing power that can only be afforded by some of the biggest tech companies around the globe [45]. Most of the advanced LLM models are owned and controlled by a relatively small group of global companies which has led to them being able to direct the use and development of the technology [45]. This coupled with the vast resources needed to develop the models constitute high barriers of entry to the market, which in turn makes competition less prevalent [45]. The heavy use or reliance of these LLMs by companies could therefore present a problem in which the users become dependent on a technology that is controlled by a few large companies who have a huge power in affecting these models.

Dependence on these models are also exposing users to having a part in other negative aspects of LLMs. One of these is the data protection and data security of the LLMs and the companies that own them. Many of the companies with advanced LLMs are located in China or the USA, countries which might not have as stringent data protection laws as, for example, the European Union [45]. This opens up the question if companies that use these LLMs are comfortable with having sensitive information input into the models. Another negative aspect of the LLMs are their environmental impacts, from their training and use [45]. Both the use and training of the models require huge computational power which is provided in the form of data centers [45]. These datacenter require a lot of electricity, which can come from fossil fuel sources, they need a lot of water resources for cooling purposes, and lastly they need a lot of natural resources to build the centers [45]

2.9 Related Work

In this section, related work and research on the subjects included in this thesis are presented. Based on the findings of the literature, a research gap is defined at the end of the section.

2.9.1 Gen-AI for Code Generation

LLMs have been shown to have great ability in processing natural language into code and many LLM models have been developed for this purpose specifically [46]. However, there are problems that can occur as natural language is a less formal way of expressing intent than code is, which can lead to the resulting code not performing as intended [46]. When prompting a LLM for code that performs a specific task, it can yield several different results that at first seem identical, but that perform differently in important ways [46]. This is often seen in *edge cases* where the natural language prompt might not have specified a preferred way to handle them, meaning that the LLM must itself define how to handle such cases [46]. For users it can be hard to catch such problems in code suggestions from LLMs as it often involves manually going through the code, which often leads to edge cases being overlooked [46].

In a study, aimed at reducing the uncertainty of LLM code generation, the StructGen framework was introduced by Wan et al. [47]. This is a multi AI agent framework that utilizes two agents with different purposes to achieve a more structured approach to code generation [47]. The StructGen framework first uses a LLM agent called a designer that, with the natural language requirements as input, creates a design scheme in the form of a Unified Modeling Language (UML) activity diagram [47]. UML is a widely adopted language in which to specify software blueprints [48]. A UML activity diagram is a way to emphasize the flow of a system from activity to activity [48]. The activity diagram focuses on the sequences of the activities, individual logic snippets, and the whole process, which makes it resemble a flow chart of how the software should work [48]. After the designer agent has provided the UML activity diagram, it is passed to another LLM agent, the builder agent, which uses the activity diagram along with the natural language requirements to generate the desired code [47]. These two agents can also work in a loop where the designer revises the design and the builder repair code through test feedback [48]. Using the StructGen framework, Wan et al. [47] found performance gains and that it outperformed competitive baselines and LLMs on public benchmarks.

2.9.2 Emerging use of LLMs for Solving MOO Problems

In recent years, the use of LLMs has increased in numerous areas of academia and industry, due to its proven and growing ability to process natural language

and solve reasoning tasks for various problems [8]. The strength of LLMs lies in being trained on massive amounts of heterogeneous data, which enable them to have strong semantic understanding, generalization capabilities and context related problem solving [38]. Along with the general tendency to explore LLMs within academia, the interest in LLM assisted MOEA has developed as a new research field. In particular, LLMs has been explored as a tool to automate and improve the demanding and time consuming design process of MOEAs, which requires expert knowledge in optimization and algorithmic design [49]. In part due to the "No Free Lunch" Theorem as described in Section 2.4.2, which forces practitioners to put considerable effort into the selection, design, and configuration of the MOEA. Although being a new terrain within the field of optimization solving, researchers have found that LLMs show promise in three predominant areas: as standalone optimizers, in low-level amplification of traditional MOEAs by embedding LLMs in the algorithm, and in high-level orchestration of selection or generation of MOEAs [38].

Taking advantage of the in-context learning capabilities of LLMs, researchers have been able to solve optimization problems using iterative prompting, making the LLM a standalone optimizer [38]. Yang et al. [50] developed the OPRO framework, i.e. Optimization by PROMpting, which solves optimization problems by iterative prompting, in natural language, that incorporate historical trajectories of solutions. In each iteration, the LLM uses previous solutions to generate new ones, in a black box manner [50]. However, Huang et al. highlights the weaknesses of LLMs in solving numerical optimization tasks, with a subpar performance when the problem lacks natural language structure [51]. Conversely, an increasing body of research suggest that LLMs might be more suited in assisting traditional algorithms contrary to replacing them as standalone optimizers [38], motivating the following research area.

In amplifying traditional MOEAs, the LLM is tasked with generating new solutions, initialization, evaluation, or assist in some operator in the EA procedure (see Section 2.4.1). Similarly to the case where a LLM is used as a standalone optimizer, LLMs are commonly used as a black box operator when embedded in a traditional algorithm, providing little insight into internal processes and reasoning [8]. In the case where LLMs are used to generate new solutions to the MOO problem, they are usually prompted by solutions from the previous generation and tasked with finding new solutions, which are then evaluated and possibly becomes part of the next generation [8]. Liu et al. [8] developed a framework for utilizing a LLM as a black box search operator

to find new solutions in the objective space. The proposed solutions of the LLM were added to the other population solutions generated by the traditional algorithm, while preserving the features of the algorithm like the crossover and mutation operations. By integrating a LLM with a traditional MOEA the authors could accelerate algorithmic convergence. The authors claim that the MOEA search strategy became more intelligent and efficient by leveraging the generation capabilities of a LLM. Furthermore, relying on the traditional NSGA-II operators, the LLM was applied only when population improvement was insufficient, limiting the interaction cost with the LLM [8]. In another study, Liu et al. [49] managed to integrate a LLM as a black box search operator, using zero-shot prompting, with a decompositions based MOEA, expanding this methodology into MOO applications [38].

Finally, in contrast to assisting at a micro level, LLMs are leveraged on a macro level, as orchestrators, to either select between existing MOEAs or to generate custom algorithms for code implementation [38]. Wu et al. [52] used LLMs to choose between algorithm by using metadata of the problem and academic papers to let the LLM extract features from the code or descriptive text to select between traditional optimization methods. In another study, Huang et al. [13] created a methodology for automatic generation of MOEAs by leveraging LLMs. Their model generate custom Python scripts of MOEA based on natural language description of the MOO problem. Furthermore, the framework iteratively repairs and adapts the algorithms through dialogue with the LLM, resulting in robust and well performing algorithms. Moreover, Yao et al. [53] developed the MEoH (Multi-objective Evolution of Heuristic) framework, which treats the generation of heuristics code as a MOO problem. In other words, LLM are used to generate a set of heuristics algorithms, which display different characteristics of e.g. accuracy and speed. The MEoH framework uses a zero-shot approach that relies on the previous training knowledge of the LLM. The framework yields a variety of heuristics algorithms from which the decision maker can apply the most fitting for the particular problem [53]. The authors suggest that their framework achieves competitive or superior performance, while improving efficiency up to 10 times [53].

2.9.3 Automation Approaches to Simulation Modeling and Optimization

Elbasheer et al. [4] present a novel methodology of using LLMs as a tool in ASMG to generate executable simulation models from natural language

prompts. The model allows domain experts to build simulations without needing the skills of specialized programming. The study showed how to enhance the human-machine interaction to allow people to build complex simulations for production planning and analysis of resource utilization, bottleneck identification and scheduling optimization [4].

In another application, Assad et al. [54] used LLMs (ChatGPT) to build a Cognitive Digital Twin (CDT) to map energy consumption and improve sustainability in a manufacturing setting. The authors describe the properties of a CDT to be dynamic, in contrast to deterministic. This means that instructions and action are generated from reasoning and processing of observed data, mimicking human intelligence, instead of relying on deterministic programming. In line with human-centric Industry 5.0, the objectives of such a system promotes human-machine interaction and knowledge transparency between systems [54]. In their study they connect a DT with a LLM (ChatGPT) to make it a CDT. The setup shows that knowledge extraction from a DES could be communicated in natural language with the help of the LLMs, improving human-machine interaction [54].

2.9.4 Identified Research Gap and Contributions

This study intend to further build upon the framework developed by Gegenmantel [14] and Schmitt et al. [16] by adding a more complex optimization layer. The intention is to create a holistic ASMG and MOO framework that requires the user to have only limited skill and knowledge of working with simulation, optimization and coding. This is done by incorporating the capabilities of Gen-AI and specifically LLMs. In Table 2.1 are listed some of the previously done work related to the study and the contributions this study provides.

The study done by Wan et al. [47] is concerned with the text-to-code capabilities of LLMs, however, did not apply it in the context of ASMG or optimization. In the study by Huang et al. [13] LLMs are tasked with generating MOEA, however, it is not applied to simulation at all. Liu et al. [8] utilizes LLMs in order to enhance MOEA, but does not connect this to simulations which is what this study explores. The study by Elbasheer et al. [4] looks into text-to-code simulation model generation, ASMG, but does not include a more advanced optimization step which this study is concerned with. Assad et al. [54] connects LLMs with a DES DT in order to get it to explain the performance of the DT, however, does not include ASMG or optimization in the approach. Lastly, Schmitt et al. [16] uses the same type of ASMG and

framework approach as this study but lacks the more complex optimization layer that this study is interested in implementing. For more information on the studies see Table 2.1.

The main contributions that this study provides are therefore:

- A holistic approach to ASMG and automatic optimization in the form of MOO by leveraging LLMs ability for text-to-code generation.
- Expanding the framework created by Gegenmantel [14] to include more complex optimization in the form of MOEA and the SCORE method for bottleneck detection.

Table 2.1: Comparison of related studies utilizing LLMs for ASMG, code generation & optimization and the gap that is addressed by this study.

Study	Domain	Simulation type	LLM role	Validation	Evaluation of LLM output	Gap addressed by our study
Schmitt et al. [16]	Manufacturing	DES	Text-to-code generation of simulation models and model output interpretation	Validated against reference model	Quantitative: code-correctness, accuracy of LLM-generated interpretations and KPIs	Did not address more complex optimization methods
Huang et al. [13]	LLM generated MOEA	-	Text-to-code generation of MOEA	Test against traditional MOEA on benchmarking problems	Quantitatively compare solutions from the code generated by the LLM	Did not include simulation based problems in the study.
Wan et al. [47]	Code generation	-	Text-to-diagram designer & Text/diagram-to-code generation	Validated against a benchmark of LLM text-to-code generation	Code correctness and accuracy regarding specified code requirements	Did not address DES code or optimization
Liu et al. [49]	LLM assisted MOEA	-	Assist in population generation for MOEA algorithm	Validated against traditional MOEA algorithms on the same problem	Evaluation of the LLM output is based on the convergence speed of MOEA algorithm	Did not connect to simulation or a automatic approach to optimization
Elbasheer et al. [4]	Manufacturing	DES	Text-to-simulation model generation	validated against traditional ASMG solutions, requiring structural inputs instead of textual	Evaluation of the correctness of generated simulation models and the time it takes to generate models	Did not include optimization step except through basic utilization driven bottleneck detection
Assad et al. [54]	Manufacturing	DES & DT	Provide cognitive capabilities to DT by providing a examination of the DTs performance	-	Qualitative evaluation of LLM textual output compared with DT performance	No focus on ASMG or specific optimization capability, investigates how LLMs can be integrated with a DES based DT
This Study	Manufacturing	DES	Text-to-code generation (simulation models, visualization diagrams and MOO workflows) and model output interpretation	Validated against reference model	Quantitative: code-correctness, accuracy of LLM-generated interpretations, KPIs and optimization results	Holistic simulation model generation and MOO automation of manufacturing systems

Chapter 3

Methodology

This section presents a general methodology to the development and evaluation of the framework that is the focus of this study. The center of attention is on describing how the framework was developed and what tools are utilized. For a more in depth description of the finished framework see Chapter 4.

3.1 Research Process

The aim of this study is to extend an existing framework for automatic optimization of a production line. The extended framework was first described by Gegenmantel [14] and later by Schmitt et al. [16]. This original framework, or workflow, gives an LLM-driven and automatic way to generate, analyze, adapt and evaluate standalone DES models, based on the Python library SimPy (see Section 3.5.2) [16]. The framework is built upon Python code with several LLM calls in order to achieve automatic execution [16]. Even though the framework have automatic components it also has human-in-the-loop aspects at several points to supervise, make adjustment and give feedback [16]. This in order to achieve higher reliability and control over the functions performed by the framework [16]. The framework utilizes a blueprint of how to set up the simulation together with provided machine performance metrics and part movement logs in order to generate the simulation [16]. The optimization is done by way of a bottleneck analysis that is performed on the current state simulation based on utilization data [16].

This study extends the earlier framework described above by modifying the way optimization is performed. In the framework by Gegenmantel [14], suggestions of improvement were derived from a utilization based bottleneck

analysis. In this project, the optimization capabilities of the framework were developed to do more complex optimization, which enables a more complete insight into how the simulation model may be improved. This more complex optimization includes MOO with MOEA as well as bottleneck detection using SCORE-analysis. After the optimization stage, both the original and extended frameworks suggests improvements that are applied to the original simulation model and evaluated for key metrics that can be compared with the original model. For a detailed description of how the extended framework is implemented see Chapter 4. This change to the framework means that it can generate a Python script with a MOEA that, when executed, optimize the production line simulation.

The framework validity was enhanced by incorporating human-in-the-loop aspects. This included visualization, validation steps (as described by [11]), and the possibility of introspection and manual changes (see Section 4.2). This feature of the framework was developed to steer the framework away from pure automation to a more engaging human-machine collaboration, as often emphasized in Industry 5.0 [11]. The human-in-the-loop stages work as both a validation tool and a way to engage the human operator to learn and participate. However, the impact of the human-in-the-loop principles were not thoroughly explored, rather they served as a guidance in the development phase.

Following the development of the framework, its accuracy and reliability, with regards to simulation model generation as well as the optimization capabilities, was then evaluated. The frameworks accuracy and reliability was assessed by its ability to generate accurate simulation models as well as its ability to run correctly without crashing or being impacted by LLM hallucination. Accuracy of the generated models were evaluated through comparisons in performance with models created in the simulation software FACTS Analyzer. The optimization capabilities of the framework were evaluated numerically by comparing the proposed Pareto-optimal solutions to results derived by the built in MOO algorithms of FACTS Analyzer. This was done by deriving certain quantitative measures to assess the quality of both solution sets in order to determine which works best. The process of evaluating the framework is described in Section 3.4.

The discussion of results will aim to place them in the context of the framework as a sociotechnical system. The term sociotechnical systems can have a wide variety of definitions [55]. In this study the term is used to describe a system where social (people) and technical aspects interact. The framework is described as a sociotechnical system in order to introduce aspects for discussion on how the technical part and the user interact with,

and influence each other. As the user plays a essential part of the frameworks function the interaction between them is an important part of the practicality and effectiveness of the framework, and puts the technical results in a larger context.

Finally, during the development of the framework, the technical work was discussed and ratified by having biweekly meetings with an expert of optimization at Uppsala University.

3.1.1 Research Paradigm

This study tries to validate a theory presented in research that LLMs can help users optimize conflicting objectives in a production simulation by developing and testing a framework for this purpose. That way the study follows a *deductive* research approach as the technical framework aims to validate established theory by applying it on a specific use case. In other words, the research pursues a "top-down" approach, typical to deductive research, i.e. beginning with the general and ending with the specific [56]. In deductive research, studies go from theory to hypothesis to data, data which can contradict or strengthen the original theory [56].

This research project builds upon established theory of MOO, Industry 5.0 and LLM-driven code generation to design a framework that implements and combines these foundational concepts to approximate traditional expert driven production line optimization. Therefore, the theory of this study is drawn from a wide range of theoretical topics, ranging from advanced optimization, to neural networks, to the role of human agency in production. Using these theories, an hypothesis was established as the foundation of this study. The hypothesis argues that the combined technologies of LLMs and optimization software could minimize the need for expert knowledge in optimization of manufacturing processes, potentially allowing lower entry barriers to solving such problems. To gather data and try this hypothesis, the developed framework was applied to a real world production line.

Furthermore, this study assumes a *positivistic* paradigm, which often is aligned with the hypothesis based deductive model of science [57]. Positivistic science generally rely on verifying hypothesis by experimentation and identifying explanatory variables or casual relationships [57]. Specifically, since the validity of this study partly relies on the quantitative comparison of the frameworks MOO solutions to the solutions generated by other simulation software, this study assumes that there are objectively optimal solutions to a manufacturing problem that can be measured and verified in a positivistic

manner. In the case of trade-off solutions such as those from a MOO problem, the objectively optimal solutions would take the form of a set of solution constituting the Pareto front.

3.2 Input Data

To create the desired DES model with the correct settings the framework need the correct input data. One of the inputs required is a production log of the events that occur in the production line. The production log input is a *csv* (Comma Separated Values) [58] file set on a standardized form, an example of such a production log can be seen in Table 3.1. This log provides data on the sequence of operations, the time of different states of the operations and the energy consumption of operations. This is used to infer information about the production line, a process which is explained in Section 3.2.1. Along with the production log the framework gets textual inputs of additional information about the production line. These additional inputs include information about buffers in the line, their name, capacity and process time. Information about defective variants are also included, the defect rate of production and how it should be handled in the model.

3.2.1 Input Data Processing

In some cases the input data needs to be processed in order to be used by the framework. This is mainly true for the production log described in Section 3.2. The production log is a standardized input so that it can be processed by a Python script in the framework. The processing includes getting the process time, availability, MTTR and energy states of the operations. For a more in depth description of the data processing see Section 4.1.3.1.

3.2.2 Validity of Input Data

Data validity is of high importance for the framework to function correctly. This is partly because of the standardized form of the production log that is required in order to compute information about the modeled system. The input information should also contain self-explanatory variable names in order for the frameworks automatic aspects, that are utilizing LLMs, to be able to make sense of the inputs. Reasonable variable names were derived through testing to make sure the framework accurately interprets them. In terms of the data

containing reasonable values for the model it therefore is reasonably simple as the production log is a log of simulated events from an existing model.

3.3 Experimental Use Case: Case A

In order to test the framework and its abilities to generate and optimize a simulation based production line, it was tested on a use case production line called *case A*. In general, case A is a simple production line of seven manufacturing operations and six storage buffers configured as can be seen in Figure 3.1.

In the development of the framework, case A was the production line that was used for assessing and modifying the Python scripts and LLM agent structures. In other words, many aspects of the framework were tailored to amend case A.

Case A was, as said, the model of a relatively simple production line in which a product variant is washed, pressed and inspected for quality issues. All the operations are of the same type of object in FACTS Analyzer, even though they are said to perform different functions. The line produces a single type of product variant meaning that no set-up times or such logic is included. The quality inspection operation has a logic that introduces a defect to the variant with a defect rate of 8.9%. The timetable object (see fig 3.1) is used to introduce a shift schedule with production stops between 17:00 Friday to 07:00 Saturday and 17:00 Saturday to 07:00 Sunday, with production happening the rest of the time. The model also includes an instance of parallel operations and an initial size for buffers as can be seen in Figure 3.1. Moreover, the simulation time for Case A was decided to be set to a simulation horizon of 8 days, with 1 day warm-up time in order for the system to reach a steady state.

To simulate case A, data is required in order to build the product flow logic and to accurately calculate valuable measurements as SEC. This is done by processing data from the production log corresponding to Case A which can be seen in Table 3.1.

3.4 Framework Evaluation and Validation

The developed framework was evaluated in two main ways. Firstly, the MOO problem solving capabilities of the framework, in which the solutions from the MOO algorithm were evaluated. Secondly, the frameworks ability to generate Python based simulation models and run without faults or crashes. These are

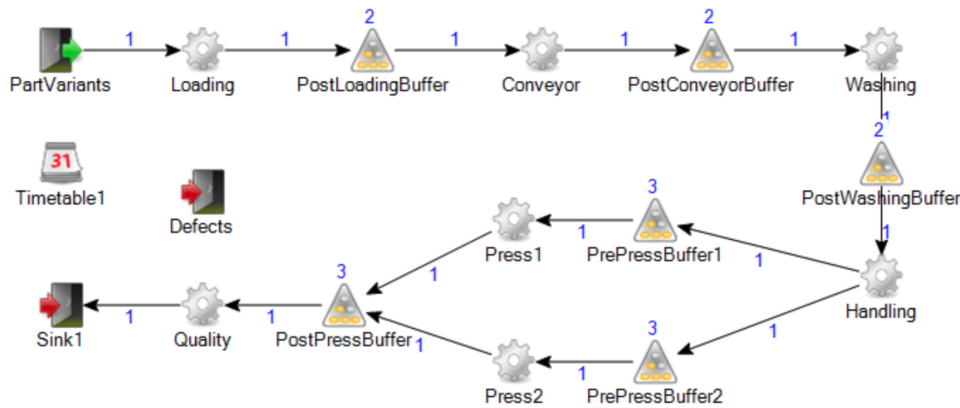


Figure 3.1: Case A modeled in the simulation software FACTS Analyzer

discussed in the sections below.

To verify the results of the framework and gain valuable insights, all quantitative results were compared to the corresponding results generated by a simulation model of case A in FACTS Analyzer. This meant that all Key Performance Indicators (KPIs) values, MOO solutions and execution times etc, needed to be verified by the FACTS Analyzer model. The results from the FACTS Analyzer model were considered to be a "ground truth", rendering valuable comparisons. However, any given simulation is an approximation of a real life system, meaning that generated results do not necessarily align with the real production line, due to their stochastic and random nature. Therefore, quantitative results from the framework model were considered valid if they, within some proximity, were similar to the data from the FACTS Analyzer.

3.4.1 Evaluation of Usability

The usability of the framework was evaluated solely by the authors and their experience in a non systematic approach. The insights gained while developing the framework and its usability are later used as arguments in the discussion and conclusion chapter for future work directives. Even though no specific scientific method was applied to test the framework for non-experts, the authors gained extensive experience on the framework which could be used to asses how non-experts could use the framework. In the study, non-experts refer to users that are familiar with production settings but might lack experience or knowledge in all or some of the areas of simulation model generation, optimization or coding.

Table 3.1: Production log for Case A: Machine operational data and energy consumption

ID	Machine	Start	End	Code	Energy (kWh)
28999b06	Loading robot	2/27/2023 08:00:00	2/27/2023 08:00:12	Working	8.64
28999b06	Loading robot	2/27/2023 08:00:12	2/27/2023 08:10:47	Idle	158.76
28999b06	Loading robot	2/27/2023 08:10:47	2/27/2023 08:11:55	Stopped	0
28999b06	Conveyor belt	2/27/2023 08:12:00	2/27/2023 08:12:06	Working	0
28999b06	Washing machine	2/27/2023 08:12:36	2/27/2023 08:12:50	Working	493.36
28999b06	Washing machine	2/27/2023 08:12:50	2/27/2023 08:31:35	Idle	4816.61
28999b06	Washing machine	2/27/2023 08:31:35	2/27/2023 08:36:04	Stopped	0
28999b06	Hantering cell	2/27/2023 08:36:34	2/27/2023 08:36:59	Working	18.5
28999b06	Hantering cell	2/27/2023 08:36:59	2/27/2023 09:31:08	Idle	1624.71
28999b06	Hantering cell	2/27/2023 09:31:08	2/27/2023 09:32:22	Stopped	0
28999b06	Presses cell 1	2/27/2023 09:33:52	2/27/2023 09:36:47	Working	224
28999b06	Presses cell 1	2/27/2023 09:36:47	2/27/2023 09:42:37	Idle	437.95
28999b06	Presses cell 1	2/27/2023 09:42:37	2/27/2023 09:43:50	Stopped	0
28999b06	Presses cell 2	2/27/2023 09:33:52	2/27/2023 09:36:47	Working	224
28999b06	Presses cell 2	2/27/2023 09:36:47	2/27/2023 09:42:37	Idle	437.95
28999b06	Presses cell 2	2/27/2023 09:42:37	2/27/2023 09:43:50	Stopped	0
28999b06	Quality station cell	2/27/2023 09:44:37	2/27/2023 09:45:18	Working	34.44
28999b06	Quality station cell	2/27/2023 09:45:18	2/27/2023 09:51:18	Idle	209.04
28999b06	Quality station cell	2/27/2023 09:51:18	2/27/2023 09:52:24	Stopped	0

3.4.2 Bottleneck Results Evaluation

The results of the bottleneck detection from the framework were validated by running an equivalent SCORE setup in the FACTS Analyzer software. The resulting frequency diagrams could then be compared and if they were similar in their results, the frameworks SCORE analysis was deemed to be valid.

3.4.3 MOO Results Evaluation

The evaluation of the MOO solutions generated by the framework were evaluated in two main ways, which are described in Section 2.4.3. The first was a visual inspection of the plotted solutions, which is automatically generated by the framework. This provided information on whether the MOO algorithm is functioning correctly and the general performance of it can be ascertained. After this initial inspection, the quality of the solutions were compared to results from an equivalent FACTS Analyzer MOEA. Here the quality difference is measured using the hypervolume QI described in Section 2.4.3. The calculation of the hypervolume for a set A of non-dominated solutions, on the objective space, is described in equation 3.1 [39].

$$HV(A) = \lambda \left(\bigcup_{\mathbf{a} \in A} \{\mathbf{x} \mid \mathbf{a} < \mathbf{x} < \mathbf{r}\} \right), \quad (3.1)$$

In equation 3.1 the $<$ denotes domination, so $x < y$ means that x dominates y [39]. $\mathbf{a}$ represents a single solution in set A . $\mathbf{r}$ is the reference point used for the calculation [39]. $\mathbf{x}$ therefore denotes all the points that are dominated by $\mathbf{a}$, bounded by the reference point $\mathbf{r}$ [39]. Lebesgue measure λ is then calculated from the union of $\mathbf{x}$ for all solutions $\mathbf{a}$ in set A [39]. Depending on the dimension of the objective space the Lebesgue measure can be intuitively understood as [39]:

- In a 2D space the Lebesgue measure calculates the area.
- In a 3D space the Lebesgue measure calculates the volume.
- In higher dimensions the Lebesgue measure calculates the hypervolume.

In this study the focus was mainly on MOO problems with two objectives. Meaning that the Lebesgue measure calculates the area of the union of all the points $\mathbf{x}$ for all the individual solutions $\mathbf{a}$ in set A . The QI value that the lebesgue measure returns is then said to be the hypervolume. In Figure 3.2 an example of how to intuitively understand the hypervolume QI for a dual minimizing problem. The larger the value of hypervolume QI, the better quality the solution set is said to be [39]. The way comparisons are made using the hypervolume QI is by calculating it for the multiple solution sets using the same reference point [39]. When using the same reference point the solution set with the highest hypervolume value was considered the best quality solution.

3.4.4 General Evaluation of Framework

In the process of running the framework, the many LLM calls generate multiple instances of code scripts and textual outputs. For this reason, many of the processes that are handled by LLMs are available for manual inspection for possible hallucinations and errors. Most notably, if any code generated by the LLMs renders any error message when executed, the fault is easily discovered for further analysis. However, if the LLM generates faulty code or algorithms that can run without casting any errors, the fault may not be found. Therefore, all data in the results section were validated in two ways, by comparing the output data to the FACTS Analyzer model, and by close manual examination

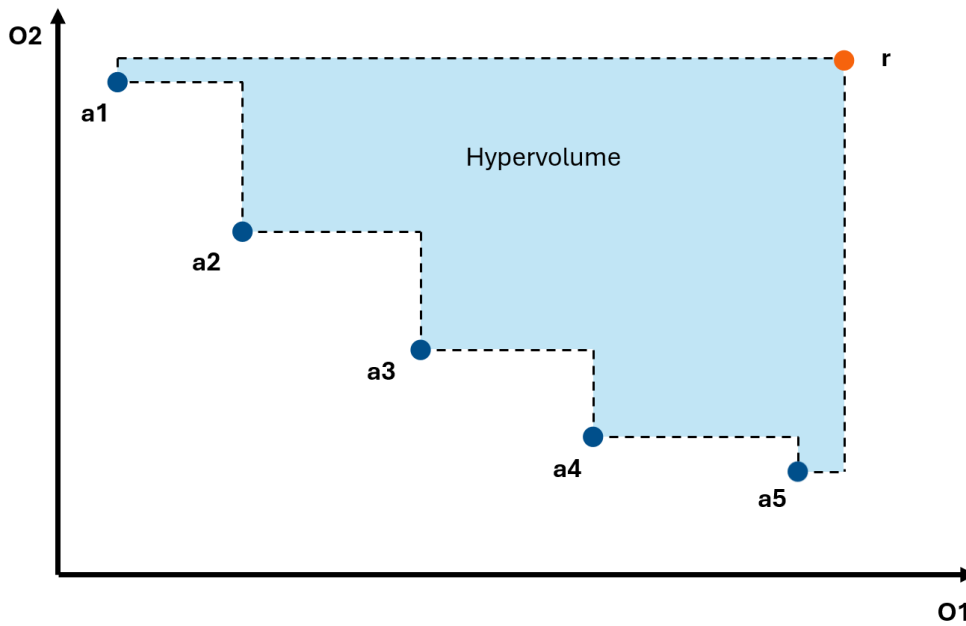


Figure 3.2: Exemplifying illustration of the hypervolume calculation for dual minimizing problem

of all material generated by the LLMs. In this way, subtle hallucinations could be detected.

All terminal outputs by the framework were also evaluated and validated based on the knowledge of the authors. For example, if the framework suggest to implement a strange solution from the set of MOO solutions, this was noted after manually controlling this step in the framework.

Furthermore, the framework was evaluated by its frequency of successful runs. This was done by simple comparing the number of times the framework managed to create useful optimization results to the number of times it did not. The usefulness of the optimization results were, as described earlier, determined by validating against the corresponding MOO data from the FACTS Analyzer model. Each time the framework produced errors or failed in some other way, the underlying reason was uncovered and described in Chapter 5.

Hallucinations were categorized in the following three categories:

- **Syntax errors:** Errors were some code script, generated by the LLMs, which could not be executed.
- **Structural errors:** Errors were code scripts that could be executed, but the generated model or algorithm did not perform its intended function.

- **Reasoning errors:** Textual outputs of the framework, generated by the LLMs, that did not accurately reflect the current state of the model, or were inaccurate in some other way.

These categories were used in order to better understand what kind of problems the framework experiences and what the underlying issue might be.

3.5 Experimental Environment

In this section, the hardware and software that were used in the project are more thoroughly presented to enhance the reproducibility of the study.

3.5.1 Hardware

In this project, the framework was run locally on a laptop with a Windows 11 operating system. The computational capabilities of the laptop mainly impacted the speed at which the MOEAs was executed, which depending on the number of evaluations and size of the MOO problem could be of significant time. The technical specifications of the computer can be seen in Table 3.2

Table 3.2: Hardware specifications of the experimental environment

Component	Specification
Unit name	Huawei MateBook X Pro Signature Edition 2020
CPU	Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz (2.30 GHz)
GPU	NVIDIA GeForce MX250 (1983 MB VRAM) Intel(R) UHD Graphics (128 MB VRAM)
RAM	LPDDR3 16 GB 2133 MT/s
OS	Windows 11, version 25H2

Importantly, all LLMs were run in a cloud setting, using OpenAI's API services [15]. Using the OpenAI API service, the LLM could be accessed from a local Python script while the computations of the neural network were executed separately in the cloud. This allowed the Python script to administer prompts and receive responses from an OpenAI LLM without requiring substantial local computational power. In the framework, in contrast to the MOEA code execution and the LLM calls, the remaining Python scripts were computationally easy and deemed not to affect the execution time of the framework significantly.

3.5.2 Software

Visual Studio Code (VSC) was used as the code editor for the framework [59], where all Python scripts were maintained. Below follows a description of the main Python libraries used in the framework as well as the simulation software FACTS Analyzer that the framework solutions were evaluated against.

3.5.2.1 Python

Python was the chosen language for the framework. It was chosen for its wide usage in academics and the easy access to resources needed for the framework such as specific libraries. Furthermore, according to research [29], Python is a coding language that LLMs tends to handle relatively well.

One of the libraries that is used for the framework is SimPy. SimPy is a DES framework that is based on standard Python [60]. The library enables users to be able to code simulation models that are flexible but does not provide a graphical user interface [60]. This lack of a graphical interface for users can mean that simulations with SimPy are less accessible and harder to produce for people who lack programming experience. In this framework, SimPy is used as the basis for all simulations.

Another Python library that is used is the MOO library pymoo. Pymoo enables Python users to get access to state of the art optimization algorithms [61]. It also provides functions for visualizing results and provide decision support to the user [61]. Together with the Simpy library the framework can create a simulation based MOO problem in Python that can perform the optimization specified by the user.

3.5.2.2 Mermaid Code

Mermaid code is a tool developed for creating diagrams and charts for documenting software workflows [62]. It is JavaScript based and uses markdown inspired text definitions when creating the diagrams [62]. Mermaid code follows UML structure as described in Section 2.9.1. It is easy to modify existing diagrams and they can be implemented as parts of scripts [62]. In this study mermaid code was used for generating diagrams describing workflows for the LLMs to emulate. As the diagrams were also generated by a LLM the ability to code diagrams made mermaid code a suitable choice. In the framework, the mermaid code was generated as *.mmd* [62] files before being compiled as *.png* [63] images for visualization.

3.5.2.3 FACTS Analyzer

FACTS Analyzer is an user friendly interactive software that is used to build, animate and execute DES systems [64]. The software also have advanced optimization capabilities which include customization of MOO algorithms [64]. FACTS Analyzer enables relatively fast and easy development of simulation models through a drag-and-drop interface with many different objects that are ready to use [64]. In this study FACTS Analyzer was used as a validation tool where the results from the framework could be tested in dedicated simulation environment with MOO capabilities. This gave a sort of sanity check of the frameworks results as well as for the generated simulation model by seeing that it performs closely to the manually built model in FACTS Analyzer.

3.6 Methodological Validity and Reliability

In these following sections the reproducibility and validity of the method is discussed.

3.6.1 Validity of Method

The purpose of the method was mainly to determine if the framework could correctly model and optimize the desired production line within an acceptable margin as well as investigating the reliability of the framework.

The validity of the method for testing if the framework could correctly model and optimize a production line was largely dependent on the comparison with the FACTS Analyzer model. This is because the correctness of the model generated by the framework was determined by comparing its KPIs to the corresponding KPIs from the FACTS Analyzer model. This could have introduced a problem if the framework model was incorrect but still produced similar KPIs, which would have made it hard find. However, this was partly mitigated by the steps in the framework where the operator is asked to verify the files generated by the framework. This way such potential problems were less likely to go unnoticed.

A potential problem with the validity of the framework as being generalizable to work with all types of production lines is that it is solely tested on a single case, Case A. This Case A is also the case that was used during the development of the framework which might make the framework biased towards working for Case A as opposed to production lines where a

different logic is required to model them. However, this study demonstrates the potential of such a framework even though it might need to be tuned in order to work with more complex production lines. This potential continuation of testing the framework on other production lines are also suggested as a future development in Section 7.3.

3.6.2 Reliability of Method

It is possible to reproduce the findings in this study by following Chapters 3 and 4, however, because of a few processes the exact result may be hard to replicate. Firstly, the code from this project is publicly available to invite any student, academic or industry professional to reproduce the results of this study. On the other hand, many aspects of the study are difficult to reproduce in retrospect because of the stochastic nature of LLMs and the EA used. LLMs are generally stochastic and do not necessarily produce the same answer in response to the same prompts. This means that reproducing the exact same outcomes is difficult even though the same prompts are used. The same goes for the simulation model in Simpy, which is stochastic by nature. Similarly, EA involve aspects of randomness, which leads to some stochastic behavior. However, these factors could be somewhat mitigated by setting random seeds in the Python code and using a temperature setting of zero on the LLMs. However, even though the exact same outputs may be hard to replicate it is possible to replicate the same behavior as described in this study by using the framework as is presented in Chapter 4.

Chapter 4

Implementation

The main technical outcome of this study is a LLM-driven framework that automatically and systematically can generate, optimize, adapt and evaluate a DES of a production line. Specifically, the focal point of this study is the optimization layer, in which settings of the simulation model are adjusted to yield the most beneficial outcomes in terms of production objectives. As previously mentioned, this project is a continuation of the work by [14] and [16], in which the authors developed a framework for automated DES model generation, bottleneck analysis adaption and evaluation. Though the utilization analysis has been replaced by an optimization layer in this study, the other aspects of the framework have largely remained the same.

In the upcoming sections, all features of the framework will be discussed in regard to their technical implementation. However, more detailed descriptions of the components not developed in this study can be found in [16] (otherwise briefly described in Sections 4.1.3.1, 4.1.3.2, 4.1.3.4 and 4.1.3.5).

4.1 Framework Overview

The framework consists of multiple core features that compromise code execution, human input and LLM-driven automatic process execution, as summarized in Figure 4.1. The framework requires a defined use case, e.g. a production line of interest, from which production data is gathered as input to the workflow. Leveraging this data, LLMs are used to generate, optimize, adapt and evaluate a DES model that represents the real life use case (see Figure 4.1). In these four steps, human input is crucial as the operator defines instructions and information that affect the prompts that are provided to the LLMs. The human operator is also encouraged to manually review and change

aspects of the simulation or optimization code generated by the LLMs. The final output of the framework is a number of implementable changes to the original model along with their corresponding KPIs. These changes could refer to, for example, changing buffer capacities or machine availability as a means to improve production objectives. Given these suggestions, a human operator can judge the KPIs of each modification and take action with the help of textual evaluations made by the LLMs. The entire process of how implementable changes are generated by the framework from the production data is described in Section 4.1.3.

In the model generation and optimization layer, LLMs are used to generate simulation code in Python's Simpy package, and optimization code in Python's Pymoo package. To reduce unwanted variance and errors in the generated code, blueprint code is used as input to the LLMs as a guiding example. This would be in line with the authors [38] and [26], who argue that one-shot prompting reduces errors in LLM generated code. The blueprint is a manually written code that exemplifies how a very simple production line can be implemented as a DES model and optimized in Python. The blueprint is explained in more detail in Section 4.1.2.

Although the framework requires a specific use case, the generalizing capabilities of LLMs, supported by [25] and [8], means that the framework can use the blueprint as guidance for a variety of different production lines. In other words, the framework is supposed to be applicable to many different production lines of similar character by changing only the input data. When a use case defined, simulations can be run and optimized automatically with human inputs that require little to no technical skill. In this study, the framework is validated by the specific use cases described in Section 3.3, simply called 'Case A'.

The full implementation of the framework in its entirety is available at Github*.

4.1.1 LLM Agent Selection

The LLM models that were chosen for the framework were instances of OpenAI models, specifically GPT-5.1 and GPT-5-mini. However, the framework is flexible in this aspect, i.e. models from other providers could also be integrated instead. OpenAI's models were chosen for this study as the original framework that this study aimed to extend used OpenAI models [14]. As such, it was deemed easier to use OpenAI models both when extending the

*<https://github.com/ViktorNorberg/LLM-DES-MOO-Modeling>

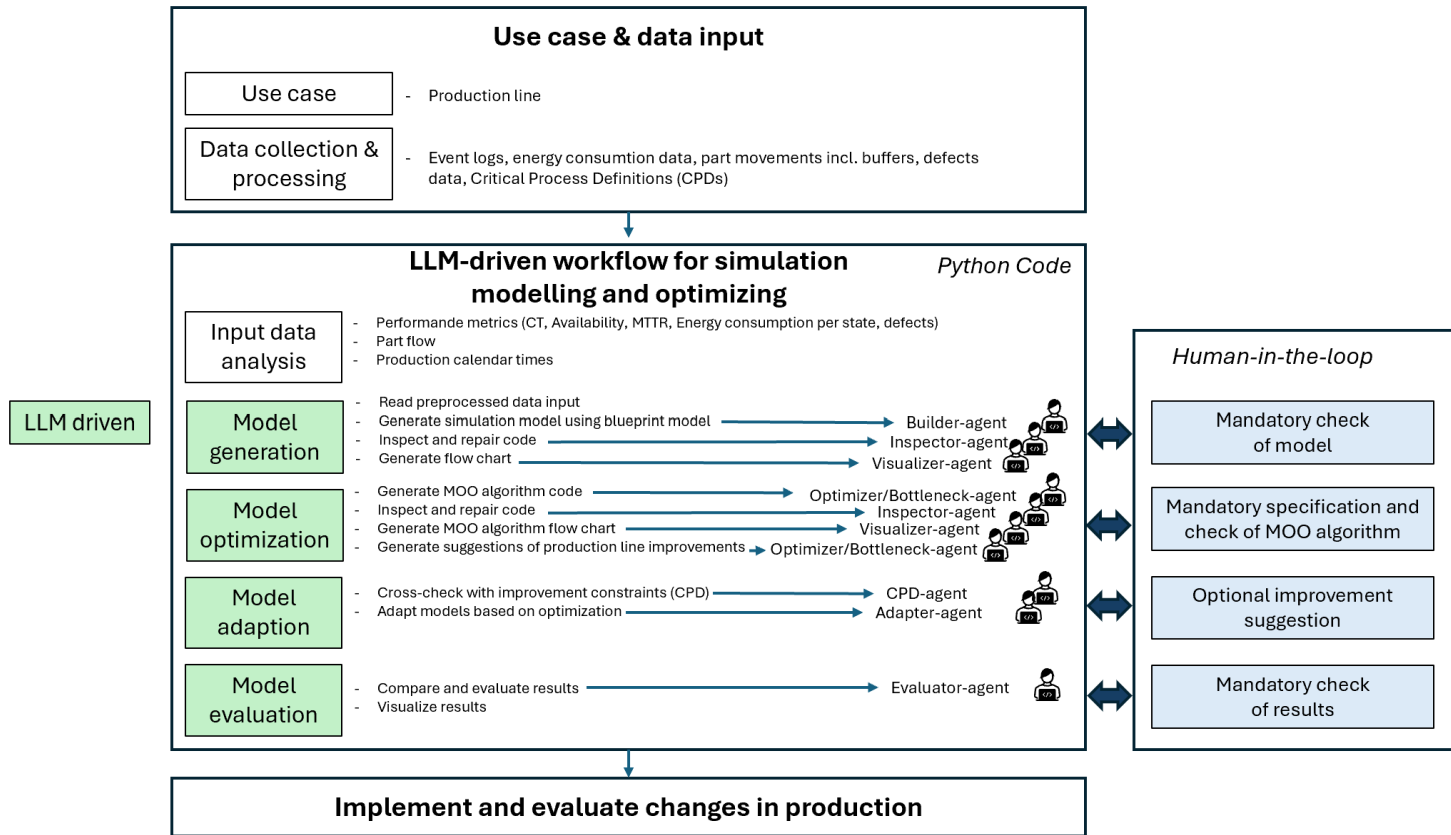


Figure 4.1: The entire modeling and optimization process of the developed framework, from the chosen use case and data input, to the LLM-driven workflow and production changes.

framework, and for comparing results. However, those comparisons are not presented in this report.

In the framework, the GPT-5.1 model is used for heavier coding tasks (see Figure 4.2). According to the web page of OpenAI [15], the GPT-5.1 model is "The best model for coding" (as per April 2026), while the GPT-5-mini model provides "near frontier intelligence" at lower cost (as per April 2026). For this reason, the GPT-5.1 was used more sparsely, when needed for complex coding, and the GPT-5-mini was used for other less complex tasks (mostly non coding assignments).

The temperature setting of the LLM calls that involved code generation and repair was set to 0.2, as advocated by [23]. The temperature setting was not derived by systematically testing the hyperparameter, instead previous

research was used as guideline. All other LLM calls had the default temperature setting, which was 1.

4.1.2 Blueprint Models

There are four blueprint models that are utilized by the framework. One of them is used as input to the initial simulation model generation and contains information on how to define and initialize objects that are needed to create the initial model objects and production line flow characteristics, as well as how to run said model. The second and third blueprints are used as input to the agent tasked with generating the code related to the MOO. Both blueprints contain the structure for a MOEA implementation, but differ on the decision variables of the algorithm. The fourth blueprint is also used in the optimization layer, but for the implementation of the bottleneck optimization. These blueprints are provided to their respective agents and provide them with a structure to follow when generating code. This is used to increase the reliability of the code generated by the agents. This means that the model building can be said to be a one-shot attempt as described in Section 2.2.2 which can be more reliable than zero-shot prompting [38] [26].

The main purpose of the blueprint code is to reduce variability in the code generation by exemplifying a working script. In listing 4.1, a snippet of the MOO blueprint code is provided which display the function that runs the optimization. In the blueprint, hyper parameters have been assigned that are important to the algorithm, but are not defined elsewhere. One such example is the crossover operator, which is set as the SBX crossover operator with a probability of 0.9. By using the blueprint, the LLMs are guided to use these settings, without acquiring the information from the human user. For this reason, the blueprint simplify the hyper parameter selection for the human operator since choices of e.g. crossover logic is not intuitive nor easy.

```
def run_nsga2_optimization(
    pop_size=50,
    n_gen=6,
    n_replications=5,
    base_seed=RANDOM_SEED,
    verbose=True
):
    """
    Run NSGA-II on the buffer capacity optimization problem.
    """
    problem = BufferCapacityProblem(
        n_var=3,
```

```

        n_obj=2,
        xl=np.array([1, 1, 1]),
        xu=np.array([10, 10, 10]),
        n_replications=n_replications,
        base_seed=base_seed
    )

    sampling = IntegerRandomSampling()

    crossover = SBX(prob=0.9, eta=15)
    mutation = PM(prob=1.0 / problem.n_var, eta=20)

    algorithm = NSGA2(
        pop_size=pop_size,
        sampling=sampling,
        crossover=crossover,
        mutation=mutation,
        eliminate_duplicates=True
    )

    termination = get_termination("n_gen", n_gen)

    res = minimize(
        problem,
        algorithm,
        termination,
        seed=base_seed,
        save_history=True,
        verbose=verbose
    )

    return res

```

Listing 4.1: Code snippet out of the MOO blueprint for a buffer configuration optimization problem.

4.1.3 Framework Stages

In the upcoming section, all the features of the framework are presented, and their functionality and logic is explained. Some example prompts are also illustrated along with some user terminal interaction when running the framework. An overview of the framework stages can be seen in Figure 4.2.

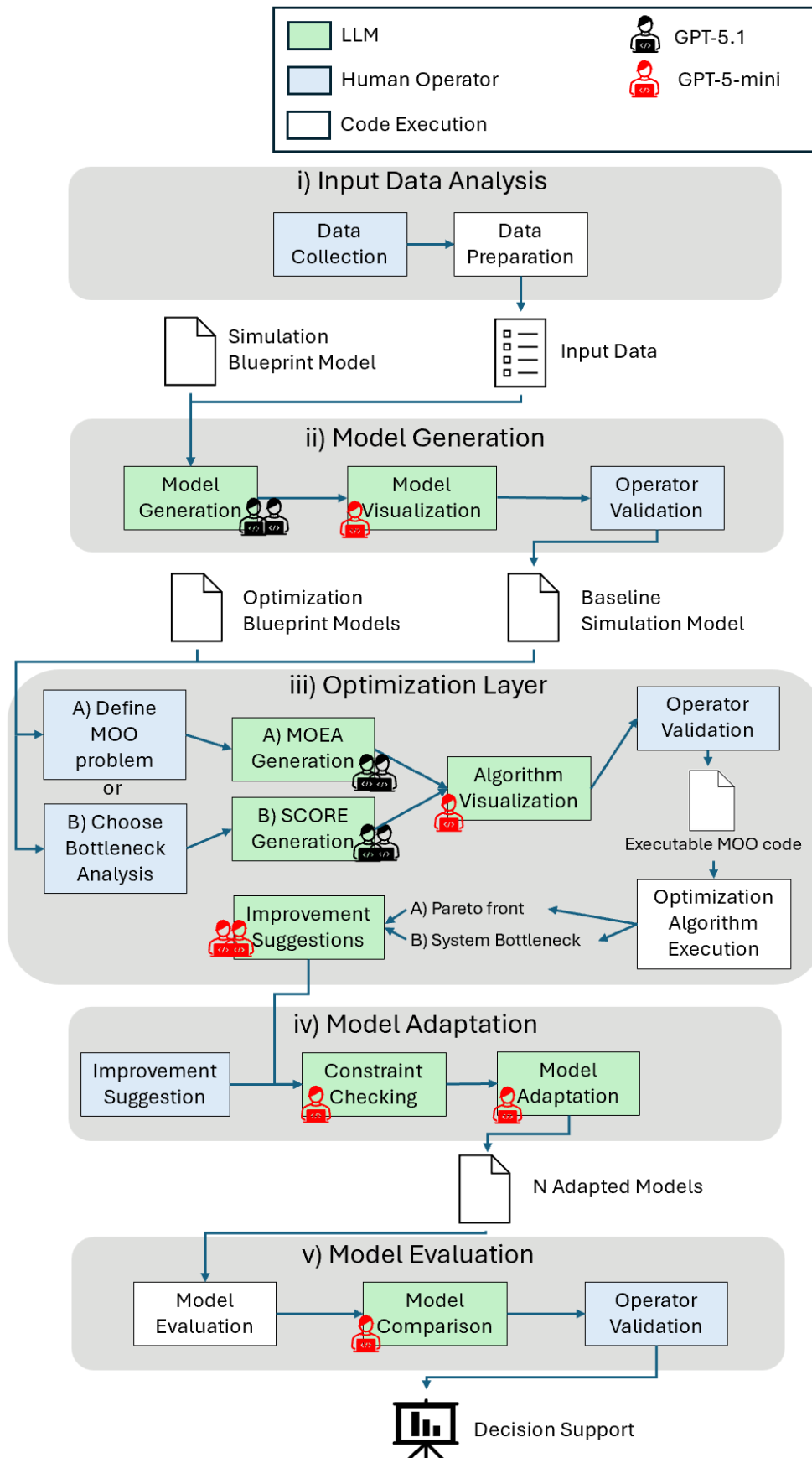


Figure 4.2: Framework overview including the five main features: i) Input Data Analysis ii) Simulation Model Generation iii) Optimization iv) Model Adaption and v) Model Evaluation. For each step of the framework, the process is either driven by LLMs, the human operator or code execution.

4.1.3.1 Input Data Analysis

The production log needs to be processed to derive necessary information to the LLMs. Firstly, the process time of each operation is calculated by taking the difference between the start and end times of the working stages. The availability is the proportion of the time where the operation was capable of producing, i.e. adding the working and idle time for the operation, and is given as a percentage. To calculate the MTTR, all instances of the operation being stopped are isolated and a mean time is calculated for these stoppages. If a machine has an availability of 100%, meaning no stoppages, the MTTR is set to 1 second as a placeholder. Lastly, the energy consumption rates for different states are calculated. The energy consumption is calculated per second and is based on the process time and energy consumption data from the production log. Information on energy consumption is also separated between the states working and idle in operations, yielding separate measures of energy consumption per second for working and idle operations.

4.1.3.2 Model Generation

The generation of simulation models is based on the work of a builder agent that takes information from the input data analysis and generates a SimPy based DES model. The code generation uses the blueprint as a guide on how to implement operations, buffers and parameterization. The builder agent receives all necessary information as natural language and its only output is an executable Python script. Before evaluating the simulation model, the code is passed for inspection, which is handled by a separate LLM call. The goal of this second LLM call is to detect any errors before the simulation code is executed.

The simulation model is run for 10 replications, each with a different random seed. To better represent the simulated lines performance a warm up period was included, based on a steady state analysis of case A. In case A, the warm up period was set to 1 day, information that is infused in the blueprint code. The simulation model collects data on the objective values of Work in Progress (WIP), TP and SEC over a period of 7 days. When the simulation script is run, at the end, the key metrics are printed and passed on as a string for further processing.

4.1.3.3 Optimization Layer

After the initial simulation model has been generated, the user enters the optimization stage where there are two possible options for optimization. The first is based on bottleneck detection with the SCORE method. The second option gives the user the opportunity to set up their own MOO algorithm by choosing from a set of predefined scenarios of decision variables and objectives.

Bottleneck Analysis

The bottleneck analysis can be run without any input by the user. The LLMs automatically generates a MOEA, according to the SCORE method, which when executed finds the five most dominant bottleneck ranked by a frequency search. The reason for this is the simplicity of the objective. Finding the bottleneck requires the SCORE algorithm to fully explore all machines of the system and their attributes of MTTR, availability and process time, meaning that the decision variables do not need to be specified by the user. Likewise, the hyper parameters of the SCORE algorithm are predefined in the blueprint. For example, the population size is 50 and the number of generations is set to 20.

The outcome of the SCORE analysis is a list of five machine properties that constrain the system the most. These five constraints are based on a frequency analysis of what constraints alleviations yield the best throughput gain, out of the solutions from the SCORE algorithm. The frequency analysis only takes into consideration the solutions of the top 5 ranks of the SCORE solutions.

At this point, the user gets to make another choice between two alternatives. In the first alternative, the five most profound constraints from the SCORE analysis are further evaluated by the adapter and the evaluator agent. The second alternative is to further investigate the most profound bottleneck by comparing different scenarios, where each scenario is a different approach of alleviating the most severe bottleneck.

To evaluate different bottleneck scenarios, the LLM agent generates five suggestions for improvement based on the modeled production line. The agent takes the simulation model and the production log data to find what buffers comes before and after the bottleneck, along with the right variable names of the availability, MTTR and process time. Five scenarios are generated accordingly:

- Increase bottleneck availability by 10%
- Decrease bottleneck MTTR by 10%

- Decrease bottleneck process time by 10%
- Double the size of the preceding buffer
- Double the size of the succeeding buffer

Subsequently, the scenarios are simulated, and the resulting KPIs are presented to the user in a plot. In this way, the user gains valuable knowledge as to which bottleneck alleviating action might yield the most beneficial improvements of the entire system.

User Defined MOEA

Drawing upon the work of [38], [13] and [52], the LLMs in the optimization process are used as macro level orchestrators of the MOEA by generating a functional Python script using the pymoo library for MOO. In this way, the LLMs choose from EO operators from a well known public resource that enables state of the art MOEAs.

Since a MOO algorithm has to be clearly defined in terms of decision variables, objectives and number of evaluations etc, it requires knowledge of MOO. For this reason, the optimization layer in the framework is scenario based, to simplify for the user. This means that the user defines the optimization problem by choosing from some predefined scenarios. Firstly, the user chooses the objectives values as a combination of two from a set of three as seen in the list below (also seen in Figure 4.3):

- Scenario 1: Minimize WIP and maximize TP
- Scenario 2: Minimize WIP and minimize SEC
- Scenario 3: Maximize TP and minimize SEC

Following the objective selection, the user is asked to select the decision variables of the MOO algorithm. The decision variables are similarly formulated as four new scenarios. In the first scenario, the buffer configuration is of interest; therefore, the buffer capacities are set as decision variables, and the user gets to choose the range of the buffer capacity. In the second, third and fourth scenario, the machine process times, availabilities and MTTRs respectively, are set as decision variables.

In Figure 4.3, an example of a scenario selection in which the user chose to minimize the WIP and maximize TP by setting the buffer capacities as the decision variable on the range 1 – 10. Furthermore, an optional limit of 50 was

Terminal Output

```
=====
-OPTIMIZATION INITIALIZED-
---OBJECTIVE SELECTION---
=====

Please choose one of the following optimization
  scenarios:
1. WIP (Min) vs. Throughput (Max)
2. WIP (Min) vs. Energy Consumption per part (Min)
3. Throughput (Max) vs. Energy Consumption per part
  (Min)

Enter scenario number (1, 2, or 3): 1

[CONFIRMED] Scenario 1 loaded:
-> Objective 1: WIP (MIN)
-> Objective 2: THROUGHPUT (MAX)

=====
---DECISION VARIABLES SELECTION---
=====

Please choose one of the following approaches to
  optimizing your selected objectives:
1. Find the best buffer configuration
2. Decrease machine process time by som chosen
  percentage
3. Increase machine availability by some chosen
  percentage
4. Decrease machine MTTR by some chosen percentage

Enter scenario number (1, 2, 3 or 4): 1

Enter the input range of the buffer capacities (e.g.
  1-10)1-10

OPTIONAL: add a constraint of maximum total buffer
  capacity (e.g. 30), leave blank if none: 50

[CONFIRMED] Scenario 1 loaded:
The MOO algorithm will use these input variables to
  optimize the objectives:
All buffer capacities. Discrete values on the range
  1-10. The total buffer capacity of all buffers is
  constrained to: 50
```

Figure 4.3: View of the user when interacting with the framework to optimize the simulation model. In this case, the human operator chose to optimize for WIP and TP by finding the best buffer configuration.

Terminal Output

```
=====
---HYPER PARAMETER SELECTION---
=====
What population size would you like to use for the
    MOO algorithm? (e.g. 100) 50
How many generations should the MOO algorithm run
    for? (e.g. 50) 10
How long should the simulation run for in seconds?
    (e.g. 10000) 691200
How long should the warmup period be for the
    simulation in seconds? (e.g. 100) 86400
-----
```

Figure 4.4: View of the user when choosing the hyper parameters of the MOO algorithm.

also set on the total buffer capacity. To simplify this scenario selection for the user, some information is already provided. For example, if the user decides to run the buffer configuration scenario, the buffer capacities are already assumed to be discrete values, an important piece of information that gets passed to the LLMs. Without specifying this, the LLMs would sometimes make the wrongful assumption that the decision variable would have continuous values.

Subsequently, the human operator is tasked with choosing hyper parameters of the algorithm, as seen in Figure 4.4. These choices let the operator control the extent of the optimization algorithm, as the parameters heavily affect the execution time.

Once the user has chosen the objectives and decision variables, the LLMs have enough information to generate the MOO algorithm that will optimize the simulation model. It does this in an ideal based approach, with the goal of finding the entire Pareto front. In listing 4.2, the prompt is shown that makes a GPT-5.1 OpenAI model generate the MOO algorithm, using the information acquired in the previous step. Moreover, the LLM receive the original simulation code, along with the MOO blueprint code. The LLM is prompted to generate the MOO algorithm, which importantly, must be compatible with the existing simulation code, since these are to be combined

after yet another LLM call. Once the MOO algorithm is combined with the simulation code, the result is a standalone executable Python script that optimizes the simulation model and outputs all the evaluations in a *csv* file format when run. The *csv* file is then used to derive the Pareto front points by a predefined script in the framework. The last step of the optimization layer involves another LLM call that chooses three points from the Pareto front as suggestions for adaptation of the simulation, which are then passed on to the adapter agent in JavaScript Object Notation (JSON) format [65]. Before picking three data points from the Pareto front, the user guides the LLM by adding any preferences. For example, if the user specifies that a high throughput is desirable, the LLM receives this information after the user makes an textual input.

```
def _generate_code(self, model_code, MOO_blueprint,
    objectives, decision_variables, population_size,
    generations, model="gpt-5.1"):
    prompt = (
        "You are an AI assistant that generates code for a multi-
        objective optimization (MOO) algorithm"
        "Your task is to generate an MOO algorithm that optimizes
        a production line simulation model in Python."
        f"This is the Python simulation model:\n\n```\n {
        model_code}\n```\n\n"
        f"These are the target objectives of the MOO algorithm: {
        objectives}\n"
        f"These are the decision variables of the MOO algorithm
        that can be adjusted: {decision_variables}\n"
        f"If a point violates the constraint, dont evaluate its
        objective values, and the datapoint should not be in the
        results csv file"
        f"The MOO algortihm should have a population size of {
        population_size} and {generations} generations. "
        "Please modify the following blueprint MOO code according
        to your instructions."
        f"```\n{MOO_blueprint}\n```\n\n"
        "Only output the MOO code, no explanations, no markdown
        fences, don't include the simulation code"
        "Make sure that the MOO code is compatible with the
        existing simulation code, and that it can be easily
        integrated"
    )
    resp = self.client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2)
```

```

try:
    return resp.choices[0].message.content
except Exception as e:
    raise e

```

Listing 4.2: Predefined prompt to the optimizer agent for MOEA code generation.

In the optimization layer, the human operator is involved in multiple steps, as seen in Figure 4.2. Firstly, the definition of the MOO algorithm is guided by the scenario selection made by the user. Secondly, the operator can validate and change the code after manual inspection, which takes place before the code is executed. Finally, as mentioned above, the user guides which data points from the Pareto front that gets chosen for further processes.

4.1.3.4 Model Adaption

Subsequently to the optimization, the suggested points of adaption are passed to the adapter agent. Though, before these are implemented, the human operator get the opportunity to add their own suggestions of improvement via text. This step can be valuable depending on the analysis of the optimization layer, or if the human operator would like to evaluate a specific point of interest in comparison. Preceding the evaluation of each adaptation, a Critical Process Definition (CPD) agent receives the suggestions in a JSON format and checks for any violations of the CPDs requirements. The CPD information is provided to the LLM agent in the form of textual descriptions of constraints to the system. For example, such a CPD constraint could be that the process time of an operation cannot be changed.

If any CPD is violated, the user is informed of this by a textual explanation in the terminal. At this point, each suggestion is implemented by the adapter agent by altering the simulation code according to the suggested change. Typically, depending on the MOO problem, the changes are parametric, and therefore easy to implement. In the case of buffer configuration, the adapter agent only has to change the buffer capacity value.

4.1.3.5 Model Evaluation

Lastly, the adapted simulation models are run and their key metrics are averaged over many replications. The resulting key metrics are plotted as a bar graph and the image saved as a *png* file. This lets the user visually look at the different model adaptations and how they perform. Then the evaluator agent

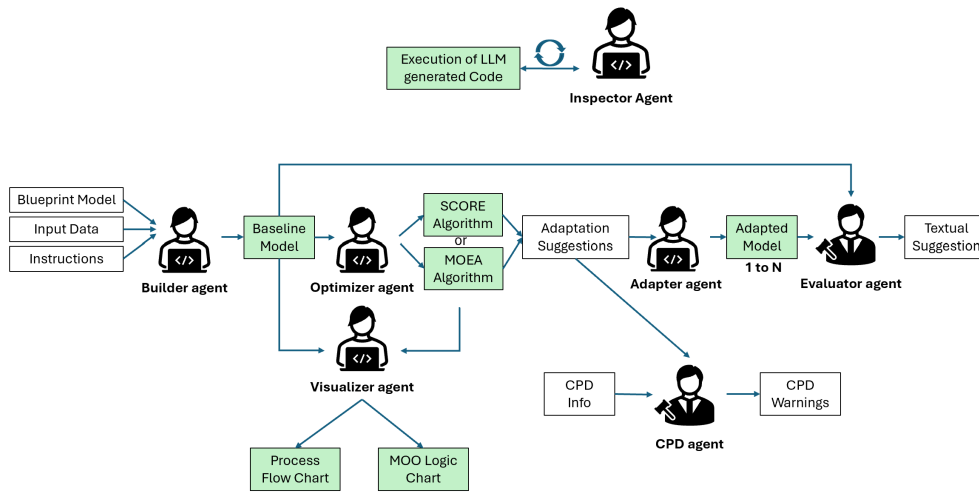


Figure 4.5: Illustration of the LLM agents and how they interact with each other. Each green square indicates that the antecedent agent has generated code which is to be executed and potentially repaired by the inspector agent.

assesses the performance of the different models, in terms of their key metrics, compared to each other and to the original simulation model. The evaluations are presented as textual outputs of recommendations that the human operator can use as guidance in decision making. In other words, the evaluator agent produces the final output of the entire framework, which are recommendations, based on key metrics and decision variable trade-offs in the production, that can be of value for the human operator before implementing any change to the real life system.

4.1.3.6 Model Inspection and Repair

In the framework, Python and mermaid code is executed on many occasions. To prevent any errors leading to the framework execution being interrupted, an inspector agent is tasked with catching any eventual error messages and repairing the code accordingly. Specifically, this is crucial at many times. These are when the simulation models, the MOO algorithm and the two visualization mermaid codes are executed (illustrated in Figure 4.5).

The code reparation is iterative, in which the inspector agent is repeatedly prompted until the code no longer cast any error. The process is governed by a 'for loop' with a maximum number of iterations, similar to the approach of [13]. The maximum iterations are set to 6. Importantly, in between each iteration, the error message gets printed in the terminal and the user gets the

opportunity to inspect or repair the code manually, or to continue the repairing loop. If the loop reaches maximum iterations, the user may try to correct the remaining code errors manually, or choose to rerun the framework. The maximum iterations are set to limit the use of OpenAI API calls in a single run. In listing 4.3, the prompt instructing the inspector agent is shown.

```
def _inspect(self, code, error_message, model="gpt-5.1"):
    prompt = (
        f"You are a Software Developer. Please evaluate the"
        f"following code and the error message from the last"
        f"execution attempt. "
        f"This was the original code: {code}"
        f"The previous code failed with the following error:\n"
        f"--- ERROR ---\n{error_message}\n-----\n"
        f"Please analyze the error and the code, and provide a"
        f"corrected version. "
        "Only output the corrected code, no explanations, no"
        "markdown fences."
        "Make minimal adjustments to the original by only"
        "changing the parts of the code that are causing the error,"
        "and keep the rest of the code intact. "
    )

    resp = self.client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.1
    )
    return resp.choices[0].message.content
```

Listing 4.3: Predefined prompt that instructs the inspector agent on how to repair the code according to the error message.

4.1.3.7 Model Visualization

To increase the understandability of the generated simulation model and the MOO algorithm, a visualizer agent was implemented to generate *png* illustrations. The intention of the images is to provide the user an overlook of the code which otherwise could be difficult to grasp. This allows visual inspection and error identification. The task of the LLM agent is to generate mermaid code, which is then used to create a *png* image. In Figure 4.6, the MOO algorithm of a specific framework run is presented as illustrated by the visualizer agent. As seen in Figure 4.5, the visualizer is prompted twice in the framework, first to visualize the production line, then to visualize the MOO

algorithmic logic as a chart.

4.2 Human-in-the-loop

The framework aims to ensure the participation of human decision makers in its workflow by integrating multiple human-in-the-loop components. The human-in-the-loop aspects of the framework are mainly concerned with setting up the simulation and optimization, validating the workflow through several interactions with the framework where feedback is provided, and validating results through manual inspection. This allocation of responsibilities ensures that the LLMs of the framework handle complex tasks, such as coding, while human operators have ongoing insight into the different steps of the framework. The integration of human-in-the-loop aspects also increase the reliability of the framework through the continuous oversight of the operator. Figure 4.1 illustrate the human-in-the-loop steps that are implemented in the framework and these are described as follows:

1. **Mandatory check of model:** In the first human-in-the-loop step the operator is prompted to manually review the initial simulation model that is generated. The operator can then modify or repair the code if needed or desired. The model generation is one of the most important steps in the framework as all other steps build on the initial model. Because of the complex code generation the agent needs to perform, this is an important step for human supervision.
2. **Mandatory specification and check of MOEA:** If the operator chooses to setup their own MOEA, he or she is asked to define important information. The operator is asked to select the objectives from a predefined list. Following this the framework asks for which decision variables should be considered and what their possible values are. Then the operator specifies the population size and number of generations for the MOEA. Lastly the simulation and warm-up time used for the simulation during the MOEA process are determined. After this, the MOEA code is generated by the framework which, similar to the first step, needs the operator to review the code manually.
3. **Optional improvement suggestion:** Here the results from the MOO problem have been generated and the framework will ask the operator to specify how the agent should consider the objectives when choosing which solutions the framework should implement, for example

prioritize low WIP. After the agent proposes solutions, the operator can also choose to manually add a solution that should be implemented and tested.

4. **Mandatory check of results:** The last step prompts the operator to review the results from the framework in order to validate them and see that they achieved the intended results.

The human-in-the-loop aspect of the framework aligns with the view of [11], who highlights the necessity that the human operator is involved in different step of the process to validate decisions and to be informed of the reasoning of the LLMs. Moreover, the visualization of the simulation model and MOO further enhances the human incitement [11]. In contrast to a fully automated system, the human-centric framework is therefore more compatible with the Industry 5.0 paradigm.

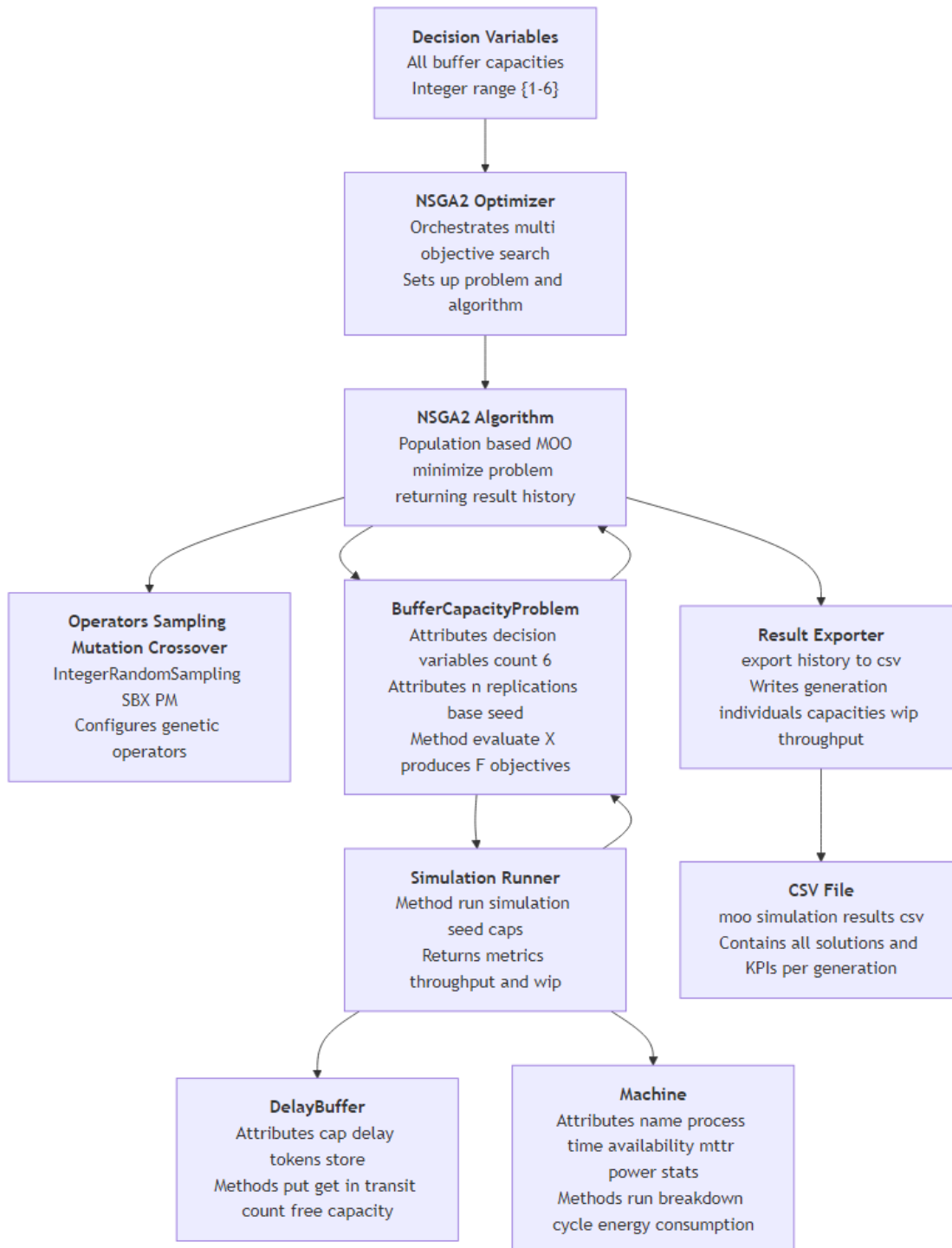


Figure 4.6: An illustration of the MOEA that was generated by the framework. Likewise, the mermaid code that rendered this .png image was also generated by framework. The image gives an insight of the main logic of the MOEA to the human operator.

Chapter 5

Results and Analysis

In this chapter, the results are presented from a buffer configuration problem and a bottleneck analysis respectively, exemplifying how the framework can assist in optimization of the production line. Following this, data from multiple runs are summarized to give insights to the general performance of the framework. Lastly, the developed framework is analyzed in context of the research questions.

5.1 LLM API Calls

To run the entire framework, a minimum of $12 + N$ OpenAI API calls are made, where N is the number of adapted models that are evaluated in the adaption phase. However, this assumes that the inspector agent is never activated. In other words, the code is generated and run without ever casting any errors. In the case when the inspector agent is activated, this adds one LLM call for each iteration of code repair, with a maximum of $4 * 6 = 24$ extra API calls. This means that the framework, in the worst case, make $12 + N + 24$ OpenAI API calls. In the optimization layer, the bottleneck detection and scenario testing requires 11 OpenAI API calls if the SCORE code casts no error. In the worst case the bottleneck needs $11 + 6$ API calls. See Figure 4.2 to see what specific OpenAI model is called in each framework step.

5.2 Optimizing Buffer Configuration of Case A Production Line

In the following example run of the framework, a scenario was used in which a hypothetical production manager wanted to optimize the performance of the manufacturing line (case A) in terms of maximizing TP and minimizing WIP by changing the buffer capacities. By choosing objectives, decision variables and hyper parameters according to figures 4.3 and 4.4, and adding a constraint of 50 on the total buffer capacity, the framework had enough information to generate a MOEA that optimized the configuration of buffer capacities. The optimization algorithm generated by the framework was set up accordingly:

- **Algorithm:** NSGA-II
- **Population size:** 50
- **Number of generations:** 10
- **Number of replications:** 3
- **Initial population:** $\mathbf{x} \sim \mathcal{U}(1, 10)$
- **Crossover operator:** SBX (prob=0.9, eta=15)
- **Mutation operator:** PM (prob=1.0/6.0, eta=20)
- **Simulation time:** 691200 seconds
- **Warm up period:** 86400 seconds
- **Constraint:** total buffer capacity ≤ 50

All algorithmic choices that the optimization agents implemented follow the operator input and the blueprint guidelines, except for the number of replications, which was set to 3 instead of the expected 5. This freedom of choice by the LLMs did not lead to any catastrophic errors, however, it did mean that each evaluation was a little less statistically certain, with the upside of a shorter execution time. In general, unexpected decisions like this example were mostly mitigated by the blueprint and a low temperature setting, which was aimed to reduce potential structural and syntax errors in generated code by decreasing variability. Nonetheless, due to the nature of LLMs, random behavior is expected.

5.2.1 Results from the Buffer Configuration MOEA

After running the optimization algorithm, the Pareto front seen in Table 5.1 was derived. In total, 19 Pareto optimal points were generated, most of them from later generations in the optimization algorithm, indicating a learning process. Notably, multiple points are duplicates, which is not uncommon for the NSGA-II algorithm. At this point, the user was asked for input on how to prioritize objectives. In this case, "best balance between wip and throughput" was used as additional information to the LLMs that was tasked with choosing three points from the Pareto front for further evaluation. The chosen data points can be seen in Table 5.2. In this case, the LLMs of the framework chose the point on the Pareto front with the lowest WIP, a knee-point, and the data point exhibiting the highest throughput.

Table 5.1: The Pareto front from buffer configuration problem of case A

Gen.	Ind.	Buf 1	Buf 2	Buf 3	Buf 4	Buf 5	Buf 6	Total	WIP	TP
1	1	4	1	1	4	3	10	23	19.15	27.58
5	12	1	1	1	1	2	9	15	12.16	26.95
5	13	1	3	4	3	4	2	17	21.15	27.58
6	19	1	1	1	1	1	2	7	11.19	26.38
7	12	1	1	1	1	1	2	7	11.11	26.33
7	14	1	1	1	3	2	4	12	14.11	27.29
7	15	1	1	1	3	2	2	10	14.15	27.40
7	17	1	1	1	3	2	3	11	14.31	27.44
8	11	3	1	1	3	3	6	17	17.39	27.45
8	12	1	1	1	1	2	2	8	11.81	26.61
8	13	1	1	1	1	1	4	9	11.33	26.39
8	14	1	1	1	1	2	4	10	11.83	26.78
8	15	1	1	1	1	2	1	7	12.11	26.83
9	13	1	1	1	2	2	2	9	13.33	26.97
9	14	1	1	1	1	1	2	7	11.11	26.31
9	15	1	1	1	1	1	1	6	11.02	26.30
9	16	1	1	1	3	1	2	9	13.36	26.99
9	17	1	1	1	3	2	2	10	14.06	27.29
9	18	1	1	1	1	1	2	7	11.14	26.34

Following the data point selection, the framework automatically generated a plot showing all evaluated points, with the Pareto optimal point in orange, and the suggested data points in green, as seen in Figure 5.1. The graph let the user visually inspect the optimization results and the data points chosen for further evaluation. The plot is displayed in a web browser. If the cursor is hovered over any point, the specific decision variable of that data point is

Table 5.2: The suggested points chosen by a LLM from the Pareto front to be evaluated further

Buf 1	Buf 2	Buf 3	Buf 4	Buf 5	Buf 6	Total	WIP	TP
1	1	1	1	1	1	6	11.02	26.30
1	1	1	3	2	3	11	14.31	27.44
1	3	4	3	4	2	17	21.15	27.58

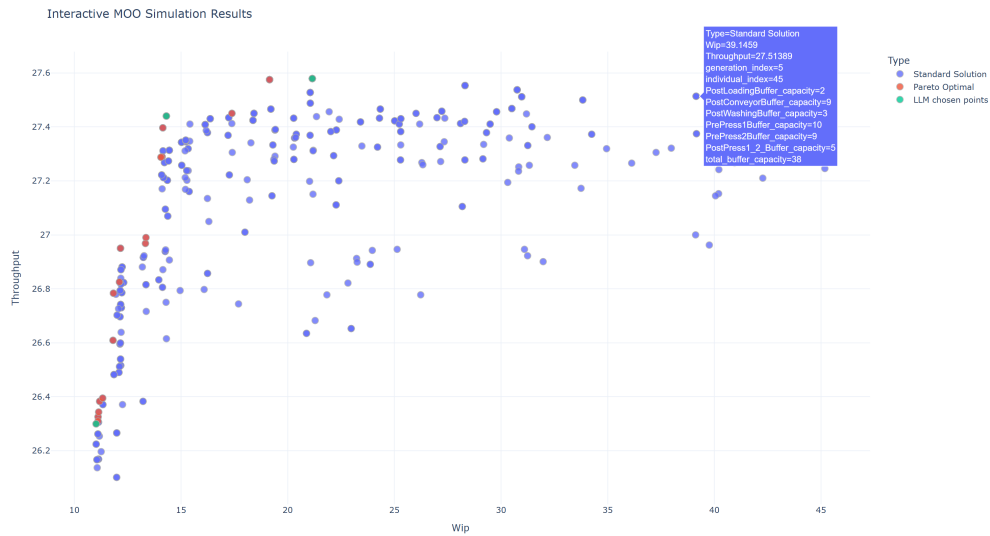


Figure 5.1: A plot that was automatically generated by the framework, illustrating the MOEA results. The graph is interactive, letting the user see the Pareto front and the decision variables values when hovering over any point.

shown in a box, allowing the user to interact with the data.

The chosen data points were then implemented as changes to the original simulation model which led to the KPI comparison in Figure 5.2.

The same MOEA was then run by the authors in FACTS Analyzer for comparison. The results can be seen in Figure 5.3. Visually, the framework algorithm explored the decision variable space more extensively, especially in configurations that yielded low WIP, while the FACTS Analyzer algorithm explored data points on a smaller space. This visual intuition was quantified by the hypervolume QI which gave the following results:

- **Hypervolume Framework:** 192.29
- **Hypervolume FACTS Analyzer:** 181.77

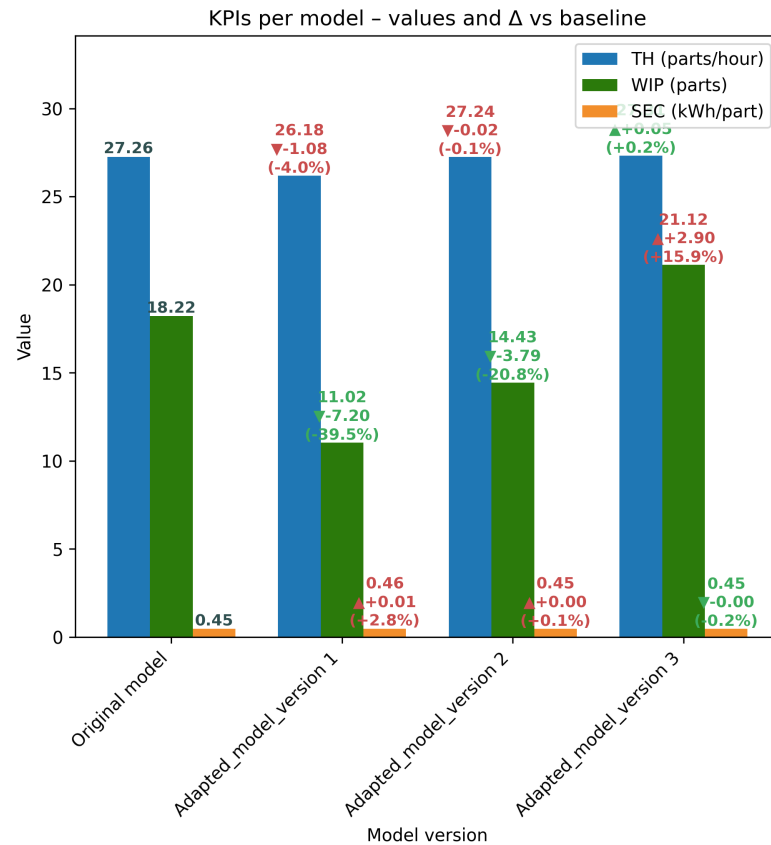


Figure 5.2: Resulting KPIs from the original simulation model adapted with the settings from the LLM suggested points. Each adapted version corresponds to changing the buffer capacities according to Table 5.2

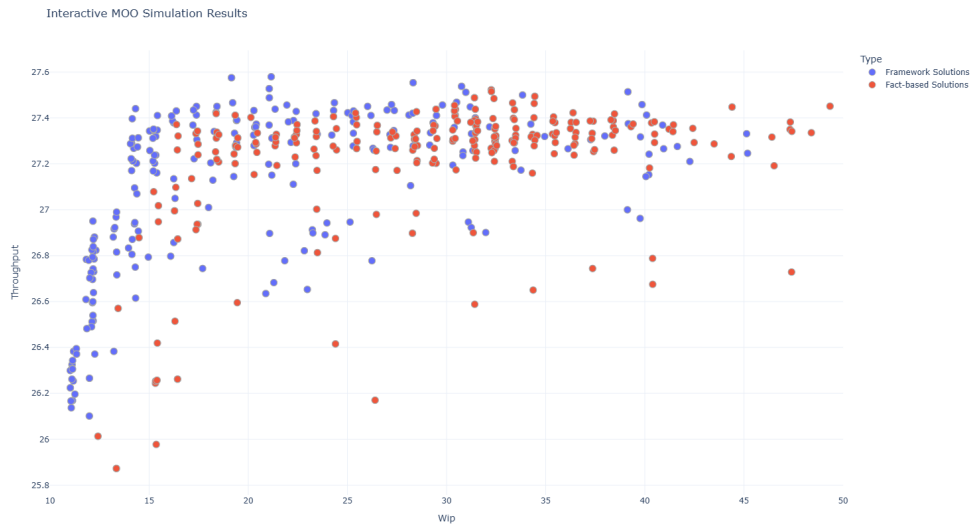


Figure 5.3: A comparison of the MOEA data generated by the framework and the FACTS Analyzer optimization for the buffer configuration problem.

Still, the hypervolumes values of the framework and the FACTS Analyzer algorithm were not that different, which indicate a relatively similar quality of the solution sets, with a slight edge to the frameworks algorithm.

5.3 Bottleneck Detection and Scenario Testing

Finding the system bottleneck

In a second example run, the hypothetical production manager wanted to find the system bottleneck and get suggestions on how to alleviate the constraint. To do so, the framework provides the option to "find the system bottleneck (by the SCORE method)." (see Figure 5.4).

In Figure 5.5, the results of the SCORE algorithm are presented as instructions on how to alleviate the top five constraints of the system. In this run, the instructions informed the user that the press operations were the main bottlenecks, and that either reducing the process time or increasing the availability were most beneficial. The instructions also came with explanations generated by the LLMs, one snippet of the explanation is shown in Figure 5.7.

When comparing the frameworks suggested bottleneck alleviations, seen in Figure 5.5, with the FACTS Analyzer frequency analysis in Figure 5.6 it can

Terminal Output

```
=====
---OPTIONAL OPTIMIZATION STAGE---
=====
Press 1 to find system bottlenecks (by the SCORE
method).
Press 2 to run specific MOO optimizations (e.g.
buffer configurations etc.).
Press 3 to exit.
Answer: 1
```

Figure 5.4: After the initial generation of the simulation model, the user gets to choose between what type of optimization to proceed with, or to exit the framework.

Terminal Output

```
{'instructions': ['Reduce the processing time of
PressesCell2 by 10%', 'Reduce the processing
time of PressesCell1 by 10%', 'Increase the
availability of PressesCell2 by 10%', 'Increase
the availability of PressesCell1 by 10%', '
Reduce the processing time of WashingMachine by
10%']}
```

Figure 5.5: The instruction for model adaption derived from the bottleneck SCORE method, exhibiting the top five constraints of the system.

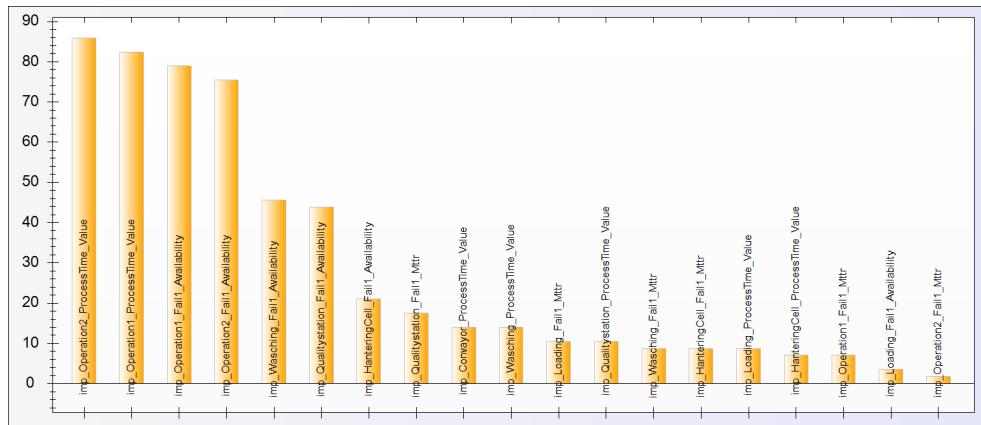


Figure 5.6: The frequency analysis from the FACTS Analyzer SCORE setup. On the x-axis is shown the improvement variables while the y-axis shows the percentage of variable improvement occurrence in solutions

be seen that they have identified similar alleviations as the most important. They have both identified the alleviations in process time and availability for Press 1 and Press 2 as the two most severe bottlenecks. However, the FACTS Analyzer SCORE method identified the Washer availability as the fifth biggest bottleneck while the framework suggests the Washer process time. This discrepancy could come from the stochasticity of the MOEA used for the SCORE method.

Bottleneck scenarios

In Figure 5.8, the different scenarios for alleviating the "press cell 2" bottleneck was evaluated. In the graph, it can be seen that scenario 2 and 3 were most beneficial for increasing the system throughput, corresponding to reducing the process time and increasing the availability respectively. Simultaneously, some suggestions did not seem to affect the system throughput, like increasing the size of the buffer succeeding the machine. This indicates that the press cell 2 is very seldom blocked while in production.

This scenario testing gives valuable insight to the user, which can see what actions have the desired affect on the system and the underlying reasons for this. In this case, the hypothetical manager could draw the conclusion that the availability and the process time seem to equally affect the throughput, and can make a decision based on other factors like cost or simplicity of implementation.

Terminal Output

```
- Reduce the processing time of Presses cell 2 by
  10%
Reason: Presses have the longest cycle times
(175 s) and are the merged bottleneck downstream.
  Making Presses cell 2 10% faster raises its
  throughput (roughly +11% capacity for that
  machine), shortens queues in the pre-press buffer
  and at the merge, reduces blocking of upstream
  machines and cuts overall WIP and waiting energy
```

Figure 5.7: A snippet of the explanations provided by the framework of the bottleneck alleviations.

5.4 Validity Analysis

To verify the results produced by the framework, the results from the buffer configuration problem in Section 5.2 were compared to the corresponding simulation model in FACTS Analyzer. The comparison in Table 5.3 show that the KPIs closely follow each other within a margin of a few percent. However, the energy consumption per part stands out as the framework consistently produces higher numbers than the corresponding model in FACTS Analyzer. Nonetheless, improving the accuracy of the simulation model generation was beyond the scope of this study and the simulation generation of the framework was deemed sufficient enough to be used for the development of the optimization layer. With that said, the discrepancy in the SEC did result in some problems for the optimization and interpose some validity problems, especially to the hypervolume QI, since the indicator degrades with a mismatch in the objective values. The reason that the framework consistently produces too high SEC values might be due to some structural error in the simulation blueprint, which the LLMs fortifies. One possible explanation could be a slight difference in how the FACTS Analyzer model and the framework determine idle and active times of the machines.

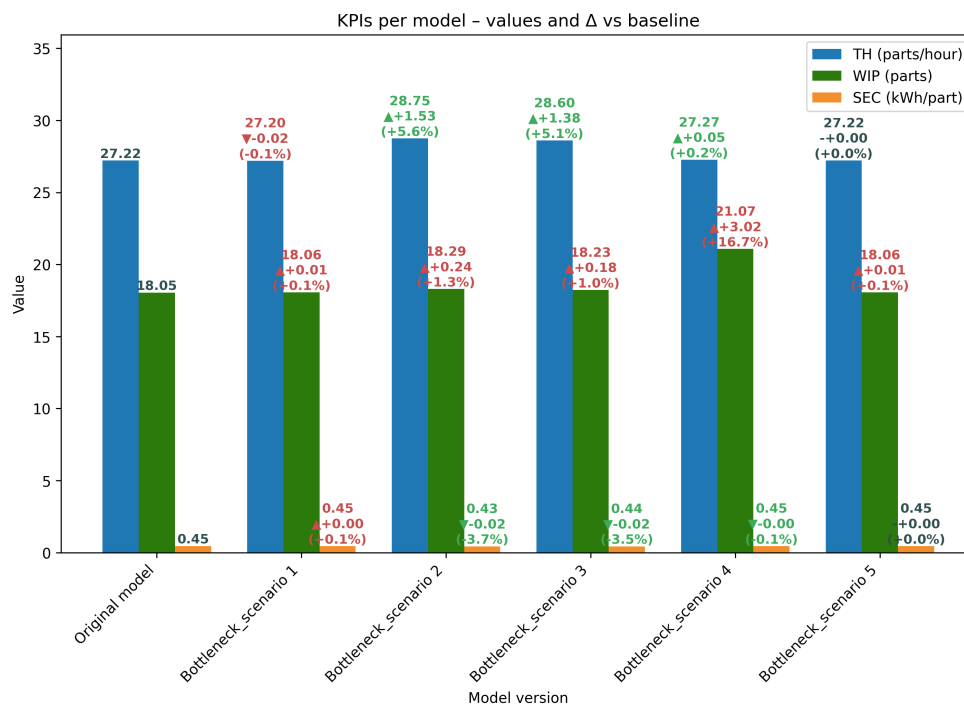


Figure 5.8: Evaluation of KPIs from the scenario testing of the system bottleneck (press cell 2). The scenarios 1-5 correspond to reducing the MTTR by 10%, reducing process time by 10%, increasing availability by 10%, doubling capacity of antecedent buffer and doubling capacity of succeeding buffer respectively.

Table 5.3: Comparison of KPIs of the framework and the FACTS Analyzer model

Model	KPI	Facts	Framework	Abs. Diff	% Diff
Initial	Throughput (parts/h)	27.31	27.26	0.05	-0.18%
	WIP (parts)	18.43	18.22	0.21	-1.14%
	Energy (kWh/part)	0.4373	0.4551	0.0178	+4.07%
V1	Throughput (parts/h)	25.99	26.18	0.19	+0.73%
	WIP (parts)	11.44	11.02	0.42	-3.67%
	Energy (kWh/part)	0.4524	0.4636	0.0112	+2.48%
V2	Throughput (parts/h)	26.59	26.14	0.45	-1.69%
	WIP (parts)	11.53	11.22	0.31	-2.69%
	Energy (kWh/part)	0.4452	0.4643	0.0191	+4.29%
V3	Throughput (parts/h)	26.76	26.19	0.57	-2.13%
	WIP (parts)	11.37	11.27	0.10	-0.88%
	Energy (kWh/part)	0.4432	0.4638	0.0206	+4.65%

5.5 Framework Performance over Multiple Runs

Another approach to evaluating the framework was a test of 10 consecutive runs. The purpose of this was first and foremost to try and see if there were any common errors that were produced by the framework. For all 10 tests of the framework, the input variable scenario chosen was a buffer capacity configuration on the range 1-6 for each buffer. No constraint on total buffer capacity was used in the test runs. Concerning the objectives of the test runs, they were divided between the two scenarios of minimizing WIP & maximizing TP (test run 1-5), and minimizing WIP & minimizing SEC (test run 6-10), as can be seen in Table 5.4. In Table 5.4, the KPIs in the model generation columns display the performance of the initial simulation model generated by the framework. In the same table, the selected KPIs in the model optimization columns was chosen by the authors as a representative point from the Pareto front of the generated solutions of the framework. The point chosen was what the authors considered to be the knee-point of the Pareto front, a common solution when optimizing for multiple objectives because of the balance it provides between the performance of each objective. In Table 5.4, the token use of each run of the framework is also reported, giving an

indication of how expensive the framework is to run. The number of tokens are a summery of both input- and output-tokens. The approximate share of input tokens were 57% of the total token use. During a test run, not seen in Table 5.4, the framework used 106 211 tokens at the cost of 0.31 U.S. dollars.

Table 5.4: Comparison of the simulation KPIs from the initial and the optimized production line in ten consecutive runs of the framework. In the right most columns, the execution time and hypervolume difference to the corresponding model in FACTS Analyzer is presented.

Run	Model Generation				Model Optimization					
	KPIs				Selected KPIs					
	TP	WIP	SEC	Objectives	TP	WIP	SEC	HV diff (%)	Time (s)	Tokens
1	27.14	18.00	0.4513	WIP/TP	26.47	12.71	0.4590	+7.15	11680	112,329
2	27.22	18.05	0.4511	WIP/TP	26.96	14.11	0.4539	+6.71	13538	105,000
3	23.21	18.15	0.5068	WIP/TP	23.17	13.23	0.5067	−50.13	7234	111,376
4	27.33	18.02	0.4274	WIP/TP	27.22	12.84	0.4293	+9.74	8887	100,246
5	23.21	18.15	0.5068	WIP/TP	23.13	14.17	0.5076	−49.40	4978	104,364
6	27.21	18.13	0.4512	WIP/SEC	27.20	13.31	0.4491	−6.72	8645	116,015
7	27.37	18.37	0.4476	WIP/SEC	27.24	14.28	0.4486	−5.19	8349	105,529
8	23.17	18.44	0.5072	WIP/SEC	23.15	11.54	0.5080	−46.62	6013	132,502
9	27.35	18.70	0.4503	WIP/SEC	27.19	11.60	0.4519	−2.82	5961	107,610
10	27.39	19.08	0.4498	WIP/SEC	27.34	12.14	0.4505	−5.30	5297	73,911

The frequency of errors during the 10 test runs is shown in Table 5.5. The table displays where each error were encountered during the framework run. Experiments 3, 5 and 8 all exhibited structural errors, as displayed in Table 5.5. The consequence of these errors in the simulation model generation can be seen in Table 5.4, as the KPIs of TP, SEC and hypervolume in run 3, 5 and 8 all deviate from the other runs. The similarity between the KPIs of the faulty models in run 3, 5 and 8 suggests that this was a similar structural error that occurred. By analyzing the simulation code of the faulty models, they seem to have had a problem with the simulation logic, specifically with the so called *helper buffers*, which were not present in these models. In the correctly functioning models, the helper buffers were used around the presses to facilitate the splitting and merging processes because of the parallel operations. The helper buffers were needed to achieve the desired function of the model in SimPy, however, not in the models created in FACTS Analyzer, which is why they are not present in Figure 3.1. Without these helper buffers in the SimPy simulation models, a logic error occurred and extra time was added to the processing time of the press operations, which in turn meant that the TP of these models were lower than the others. Importantly, even though the simulation model produced faulty throughput values, the simulation models

were still sound enough to be used for further evaluation of the optimization capabilities of the framework. This meant that good trade-off solutions could be found and that the optimization layer of the framework still worked as intended.

Table 5.5: Accounting of produced errors from ten consecutive runs of the framework

Run	Model generation		Optimization		Model adaption		Model evaluation
	Result	Error	Result	Error	CPD	Result	Result
1	✓	—	✓	—	✓	✓	✓
2	✓	—	✓	reasoning	✓	✓	✓
3	✓	structural	✓	reasoning	✓	✓	✓
4	✓	—	✓	—	reasoning	✓	✓
5	✓	structural	✓	—	✓	✓	✓
6	✓	—	✓	—	✓	✓	✓
7	✓	—	✓	reasoning	✓	✓	✓
8	✓	structural	✓	syntax	reasoning	✓	✓
9	✓	—	✓	—	reasoning	✓	✓
10	✓	—	✓	—	✓	✓	✓

Looking at Table 5.5, there are three reasoning errors labeled in the optimization part. These errors affected experiment runs 2, 3 and 7. The label reasoning error is given to an error where the authors deem that the textual output or non-code related outputs of one of the LLMs agents were inaccurate or unsatisfactory. In the case of reasoning errors seen in Table 5.5, it refers to when a LLM agent was prompted to choose 3 points from the Pareto front for further evaluation. In the case of these errors, the chosen points were deemed to not be the best choices. In Figure 5.9, the generated solutions from experiment run 2 are shown as an example of such a reasoning error. In Figure 5.9, one of the LLM selected points were close to the knee-point region of the Pareto front. However, in run 2, the LLM had selected a point that was at an obvious gap in the Pareto front, meaning that if a complete Pareto front of solutions was to be generated, this selected point would not be on it. Therefore, the selection of this point was deemed as a reasoning error by the LLM. This result further strengthens the argument of [30], who points to the weakness of LLMs when directly prompted for decision support, which is the approach of the framework. The user inputs some preference in free text, which could be more or less structurally sound. In other words, there is no guarantee that the LLM receives enough information of goals and context to make well balanced

decisions.

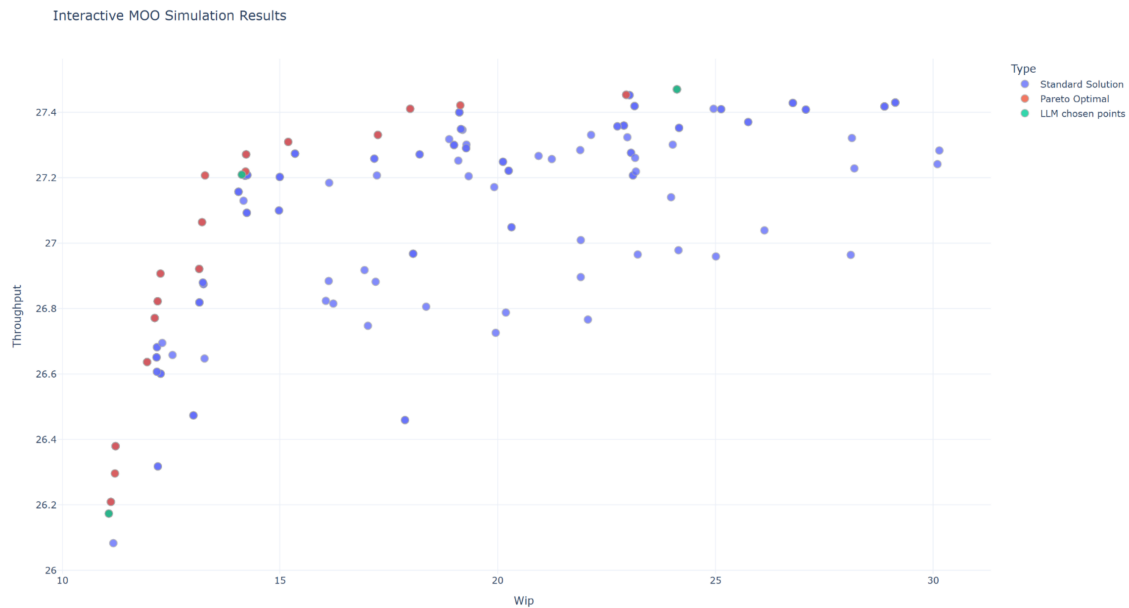


Figure 5.9: Example of reasoning error from LLM when suggesting points from Pareto front

There are also reasoning errors listed in the model adaption CPD stage, as shown in 5.5. This refers to the CPD agent not picking up a constraint violation that is being suggested by the optimization stage of the framework. During the testing of the framework on the buffer configuration problem, a CPD constraint was added that stated that the buffer capacity must stay the same level for all buffers. As the tests were on buffer configurations, the CPD agent should have picked up this violation for all suggested adaptations to the model. In the case of experiment run 4, 8 and 9, this was not the case and the CPD agent only gave a warning for 2 out of the 3 suggested adaptations, even though all suggestions involved changes to buffer capacities.

In course of the 10 runs, only 1 syntax error was generated by the framework as seen in Table 5.5. In run 8, the `def__init__` function of the optimization problem was wrongly setting up the lower and upper bounds of the decision variables, resulting in the following error message:

```
”[...] self.xl = name: var.lb if hasattr(var, "lb") else None for name, var in
vars.items() AttributeError: 'list' object has no attribute 'items'”
```

When the Python script was executed, the error message was caught and sent to the inspector agent. Together with the erroneous code, the inspector

agent repaired the code in one iteration, and the flow of the framework could then continue like normal.

In Table 5.5, the hypervolume difference indicates that the framework MOEA performed better than the corresponding algorithm in FACTS Analyzer when the objectives were set to WIP and TP, while performing worse when the objectives were WIP and SEC, with the exceptions of runs 3,5 and 8. The NSGA-II algorithm in FACTS Analyzer was designed to mirror the one that the framework generated. However, some settings were not accessible, for example the η hyper parameter of the crossover and mutation operators. For this reason, there might be slight differences in the algorithms, which could be one explanation to the difference in hypervolume. Nonetheless, due to the stochastic nature of NSGA-II there will always be the aspect of randomness in the solution set. Interestingly, the framework outperforms FACTS Analyzer with a similar margin in run 1,2 and 4, which might indicate that the framework sets up the NSGA-II better for this particular optimization problem. As mentioned earlier, the "No Free Lunch" theorem states that no MOO algorithm will outperform all other algorithms for all optimization problems [38]. Therefore, it cannot be said that the framework will consistently outperform the NSGA-II of FACTS Analyzer, especially considering the small number of test runs. Nonetheless, the hypervolume indicator gives an indication that the framework is able to produce sufficiently good trade-off solutions on the Pareto front.

In the case where WIP and SEC are the optimization objectives, the FACTS Analyzer algorithm consistently outperforms the framework. This is likely due to the fact that the SEC values produced by the framework simulation tend to be higher than in FACTS Analyzer, as discussed in Section 5.4. In Figure 5.10, the solution sets of run 6 clearly illustrate the difference in SEC. This difference impacts the hypervolume indicator to the advantage of FACTS Analyzer.

As seen in Table 5.4, one of the columns show the time taken for the execution of the optimization algorithm. There is a considerable variation, ranging from 4 978 to 13 539 seconds. Moreover, even the fastest run is much slower than the same optimization in FACTS Analyzer, which typically took a little more than 100 seconds. The time discrepancy could be due to multiple reasons. Primarily, the blueprint codes used to guide the LLMs in simulation generation are not necessarily designed for efficiency and speed, which would mean that the framework generates code that is not fitted for fast evaluations. Secondly, the FACTS Analyzer software is a purpose built simulation engine. This could mean that in general, the FACTS Analyzer simulation engine is

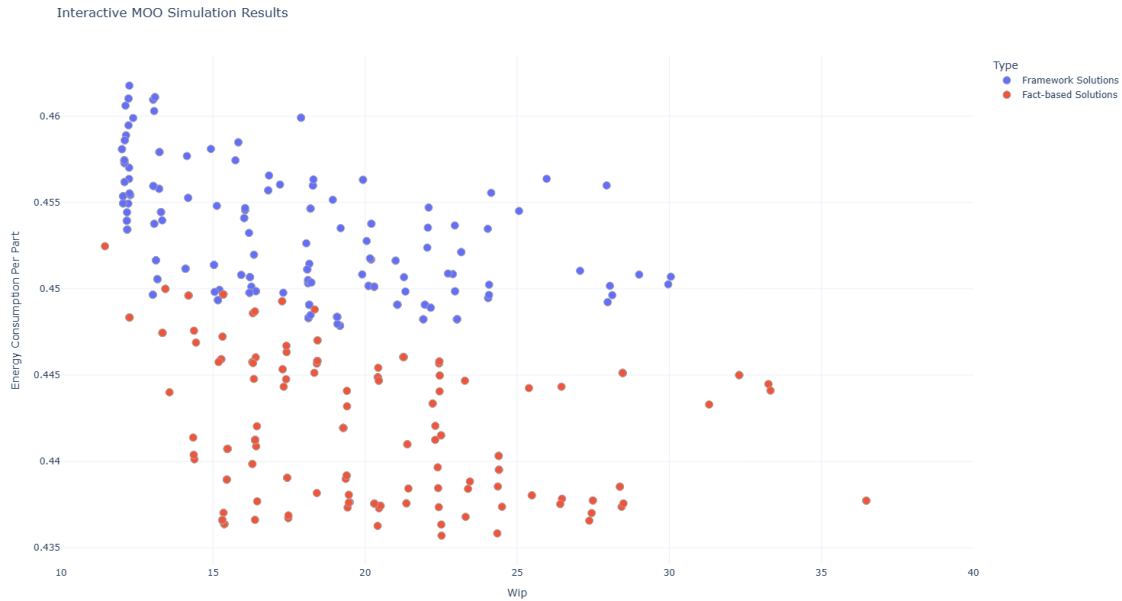


Figure 5.10: Graph of the MOO results with both the framework and FACTS Analyzer solutions. The data is from run 6 of the multi run experiment in Section 5.5.

more efficient than the one of SimPy, and even more so the blueprint code of the framework.

The large variety in execution time of the framework could be due to the randomness that the LLMs introduce. Depending on the individual run, the generated MOEA, and simulation model, could be more or less efficient. The long run times are most likely explained by the simulation model, however, the variety in time could be due to both the simulation model and the structure of the MOEA.

5.6 Analysis of the Results

Looking over the result, it seems that a LLM-based human-in-the-loop framework can be used to optimize conflicting objectives in a production line by generating a simulation model and a MOEA. The framework is able to create a valid simulation model as well as optimizing it using MOEA, both as a scenario based MOO problem and as a bottleneck detection using the SCORE-method. Because of the focus on non-expert users, the human-in-the-loop aspects were structured to allow for a relatively small amount of interaction to

still yield valuable simulation and optimization set-ups. However, the multiple human-in-the-loop validation steps also allow the user to interact with the generated code, which means that a more experienced or knowledgeable user can still achieve increased customization. In comparison to FACTS Analyzer, the framework allows the user to have full insight into the source code of both the simulation model and the optimization algorithm. In other words, the framework enables introspection and verification of the results, increasing the validity of the framework. In other words, the "black-box" problems of the LLMs [28] is reduced by the structure of the framework. The LLM do not generate results themselves. Instead they generate code that produces the results, which increases the transparency. This aspect of the framework highlights the human agency and makes sure that the human operator has access to decision making reasoning. This is an important aspect of human agency in the human-machine collaboration that is highlighted in Industry 5.0, as mentioned by [9].

As for the framework producing valid simulations and optimization algorithms the results seem to confirm this. The initial simulation model KPIs are relatively closely matched with the KPIs of the FACTS Analyzer model, which suggests that the framework produce valid simulation models. The quality of the solutions generated by the frameworks MOEA also seems to be relatively closely matched with FACTS Analyzer, based on the hypervolume comparisons of the solution sets.

However, in some aspects the framework might not be said to achieve a similar performance to the FACTS Analyzer model. The runtime of the MOEA in the framework is considerably longer than in FACTS Analyzer. On the other hand, if the total time spent to achieve both optimizations is measured, one might argue that the framework is relatively time efficient. To manually create a simulation model in FACTS Analyzer can be time consuming, while the framework takes a relatively short time to generate a model. Furthermore, it takes skills or experience to build a simulation model in FACTS Analyzer. However, the act of creating the FACTS Analyzer simulation model is also a one-time time expense, after the initial creation of the model each optimization run requires little time.

Furthermore, the framework does experience reliability issues. As seen in Table 5.4 and 5.5, the LLMs does suffer from hallucinations in which structural, syntax and reasoning errors appear in the framework. For a human user, this would mean that the framework would not reliably produce good result, and therefore could lead to unfeasible or poor improvement suggestions to the production system. Notably, structural or reasoning errors could pose a

real issue to the user if they lacked experience in coding and or optimization, since the framework would run without casting obvious syntax errors. The faulty logic or reasoning could be difficult to detect for a non-expert user, and might not be discovered, potentially leading to unsound production changes. For example, in run 3 (see Table 5.5), the simulation model had a lower throughput than expected, the operator would likely notice this but might not be able to change the simulation code to correct the error. Instead, the user would have to restart the framework, using up more valuable time. Moreover, if the user relies on the textual explanations in the terminal, due to being a non-expert, any reasoning error made by framework could lead to false inference by the user. This would align with what [28] argues, that ill informed users might put too much trust in the answers from a LLM, while an expert in that area would more likely recognize faults.

Chapter 6

Discussion

In this chapter, a technical discussion is held in regards to the results in Chapter 5. Moreover, some technical aspects of the framework are discussed that were not mentioned in previous chapters. These include design choices and general learnings from the process of developing the framework. Following the technical discussion, some sociotechnical reflections are made on the possible social implication of a system like the framework presented in this thesis.

6.1 Technical Discussion

As already mentioned in Section 5.6, the framework is able to produce valid simulation and optimization results that are consistent with the validation model built in FACTS Analyzer. One of the main problems with the framework can be said to be its reliability. In Section 5.5, the reliability of the framework is described and the different errors presented. From this we can see that the framework is not always reliable. It is of course a problem that the framework does not produce the correct output, but the types of errors that are created might be especially detrimental to the user of the framework. This is because one of the most common errors encountered was a type of structural error in the model generation stage. As described in Section 5.5 this error did not impede the model from running, but would produce erroneous KPIs. This type of error can be especially critical because it does not enable the iterative inspector agent to fix the code (as it does not produce a syntax error), and it must therefore be spotted by the user. As the intended user is a so called non-expert, this might cause the error to go without notice and therefore cause a faulty optimization to be the support for decisions made by the user.

On the other hand, such errors should still be relatively easy to spot. In the case of this study, the validation model in FACTS Analyzer acted as the guard against accepting KPIs from a faulty simulation model. A user may not have that, but, they would have data from the real production line that they are trying to model and optimize using the framework. This data could therefore be used to validate the model generation step of the framework, ensuring that models with structural errors are noticed.

Reasoning errors in the optimization stage can occur, in the form the framework choosing under-performing settings, as described in Section 5.5. This could be hard to detect for a user if they are not careful. That is why the human-in-the-loop inspection and validation steps of the framework are important and need to be utilized by the user to achieve the best performance. When the MOO solutions are presented to the user, it is possible for them to see such reasoning errors and act accordingly.

Still, if the framework often produces these errors, a user might still feel that the framework is unreliable and choose not to use it. For this problem, the only real solution would be to continue developing the framework to increase its reliability.

Though the framework faces some challenges in terms of reliability, it does contribute to the identified research gap as addressed in Section 2.9.4, by providing an example of how LLMs can be used to automate advanced optimization in manufacturing in a human centric approach. Specifically, the framework created by Gegenmantel [14], was expanded to allow MOEA and SCORE method optimization. By doing so, the research field of ASMG, MOO and AI were combined to advance the vision of non-expert accessible simulation based optimization. This statement is supported by the results in Chapter 5 of the study that demonstrate how valuable insights of the production line case A, in terms of finding bottlenecks and optimizing buffer configurations, could be achieved by little to no user input to the framework that could be considered to require expert level knowledge. The results indicate that LLMs might be a well suited technology for further research how to best support non-expert human operators in industry who are tasked with production line optimization.

6.1.1 Insights from Development Stage

During the development of the optimization layer of the framework, multiple different approaches were tried and tested before arriving at the current version of the framework, from which the results were obtained. With this in mind,

many insights gained during the process of developing the framework have not been analyzed in the previous chapters. In this section, a discussion of these insights follows. Importantly, the following arguments are not supported by any empiricism except the experience of the authors from working with the framework. For this reason, none of the discussions below are to be considered the scientific result of this study.

Firstly, at an early stage of the project, the optimization layer generated MOEA code without the help of any blueprint Python script. In general, this meant that the generated MOEAs showed more variety, and a higher frequency of structural and syntax errors. Even though the code could be repaired by the inspector agent, this made the framework difficult to work with, since it regularly had to be restarted or manually corrected to produce results. Introducing a MOEA blueprint drastically improved the performance of the framework, by decreasing errors in the code generation, much like the authors of [38] and [26] argue one-shot prompting is more reliable than zero-shot prompting.

Secondly, lowering the temperature setting of the LLM calls, tasked with code generation, further decreased the variability and the number of errors produced. Initially, the temperature setting was not specified, meaning that the temperature was set to 1 as by default. The default setting works perfectly fine for most LLM tasks. However, for coding purposes a lower setting tended to yield better performance, a tendency that the authors of this study experienced and is also argued by the authors of [23].

One could argue that the use of a blueprint and low temperature settings might lead to LLMs that only recycles predefined code, while being restricted to find potentially better solutions. This could make the framework rigid and not adaptable enough to be able to handle deviance in the user inputs. While this argument might be true, the framework does not claim to be applicable to all sort of optimization problems, instead it is based on optimization scenarios. In the scenario approach, the reduction of variability leads to better performance in terms of fewer errors. However, if the user makes nonsensical inputs to the framework, or request optimization procedures not similar to the blueprints, the outcomes might vary. These choices were aimed to increase the reliability of the framework, so that it could consistently produce good trade-off solutions for the user to contemplate. Without consistency, the framework would quickly be regarded as unusable, as it would be frustrating and time consuming to rerun it. For these reasons, the decision to continue with the blueprint and the low temperature approach was deemed valuable to the overall performance of the framework, even though it might have lost some of its

flexibility.

6.2 Sociotechnical Reflections

In this section, some reflections are discussed about the sociotechnical aspects of the framework.

6.2.1 Keeping Humans-in-the-loop in the Advance of AI

The framework could be considered a human-machine interaction technology where the framework is a tool used to extend the capabilities of the operator. This is a way to incorporate the capabilities of LLMs while preserving a human centric perspective, which is an important part of the thinking related to Industry 5.0, as described by [9]. Therefore, the human-in-the-loop aspects of the framework makes it more than an strict automation approach to simulation and optimization. This is done by requiring the interaction and validation of a user, but still simplifying the process enough to make it accessible to a wide array of users.

As a tool that a human user interacts with, the framework have sociotechnical aspects that merit a deeper discussion. There has long been discussions regarding machine and human competition in workplaces as technology has evolved. This is especially true when machines can automate tasks that were previously done by human operators and so replaces them in the workplace. This has led to loss of jobs and also deskilling of work as argued by [43]. This trend has previously been confined mainly to work that involves physical labor but has, with the development and diffusion of Gen-AI, begun affecting many people that do cognitive work as well, one example being decision making that is outsourced to AI [43]. This framework could be seen as contributing to this trend by enabling access to simulation generation and optimization to people without requiring much background och knowledge within either subject. It could therefore be said to exist a tension in the use of such a framework as presented here, between providing the benefits of simulation based optimization to more people and the trend of deskilling cognitive work.

There might also be a case for arguing that a framework such as this, because of the deskilling aspects, is part of the more long-term problem of limiting learning opportunities for future experts of the subject, as argued by

[44]. Even though the entire simulation model generation and optimization stages of this framework might not be considered an entry-level activity, there are still functions of the framework that can take away such entry-level processes. There will probably always be a demand for experts in the simulation and optimization fields. Since even if a technology such as the framework is used, there are functions it can not perform. The framework presented in this study is scenario-based, meaning that the user can choose between a few predefined scenarios of optimization, making it easy to use without much experience. This makes the optimization part unable to be configured exactly like the user wants, but if it framework was changed to allow such for configuration it would require vastly more knowledge of simulation and optimization to operate. This might mean that the work that could be replaced by the framework in its current form is of relatively simple tasks, making it more susceptible to contributing to the long-term decline of experts in the field by removing learning opportunities for junior operators.

Both the problems described above, deskilling and automation of learning opportunities, have in some ways been addressed by the framework by its extensive human-in-the-loop aspects. By allowing the operators to interact with and validate the work done by the framework it allows for a greater understanding of the functions it performs. It also makes it easier for users to detect errors and understand the reasoning behind the decisions of the framework, partly because of the multiple visualization steps. By doing this, the framework can be seen as more of a tool that is used by an operator who is forced to, at least to some degree, interact with and understand what the framework does. This might decrease the risk of decision-making skills being totally outsourced to AI and automation as brought up in Section 2.8. This would then in some way ensure that the human agency over the system is preserved while still providing relevant information to support decision making regarding the modeled production line.

6.2.2 The Cost of Technological Dependency

To use the framework that of this study it is necessary to have access to LLM models of good performance. This leads to some potential problems that can arise from being dependent on this technology, some of which are described by [45]. One of these problems is the data security and protection that they entail. A production company looking to use the framework might be unwilling to input information of their production line to the LLMs as they might be afraid that the information could be accessed by others. Problems such as this one

can also be hard to negotiate around with the companies who own the LLMs as they are relatively few and hold a very strong position in the market, this lack of competition mentioned by [45] might make them less accommodating to such request as there are few alternatives to turn to. Another potential problem that a company might consider is the environmental and ecological implications of using the models, which require many resources to train and run, as laid out by [45]. These environmental aspect might make a company less interested in using LLMs. However, in the context of the framework as described in this study, the use of LLMs might be considered small in absolute terms. Still, the increased use of LLMs contribute to the problems mentioned and should not be trivialized.

6.3 Framework Applicability in Industry

The framework could potentially be utilized by industry as a means to overcome some of the challenges that are highlighted by [18]. One of those challenges is achieving higher efficiency while decreasing environmental and ecological impact [18]. The framework allows the user to optimize these aspects by minimizing SEC while also optimizing either the WIP or TP. However, optimizing for lower energy consumption is not anything new, rather the framework promotes easier access to this technology which can make the barrier to produce in a sustainable way less intimidating. Nonetheless, the SEC KPI does come with some drawback since it only measures the average energy consumed for each produced part. For this reason, only optimizing for low SEC can lead to over production, since a higher throughput decreases the SEC. This could be even more detrimental for the environment since the total energy consumption could increase.

Furthermore, [18] mentions the need for automation tools to be able to handle dynamic conditions, high complexity and fast development cycles to be useful. The framework could be said to fulfill at least two of these. Firstly, it promotes fast development cycles since the simulation modeling is fast due to not requiring any manual simulation modeling, in contrast to traditional software like FACTS Analyzer. Therefore, it can be applied to many different sub lines in a flexible and quick approach, only requiring a production log of that specific line. The idea that LLMs can reduce development cycles of simulation modeling is argued in a similar way by [6]. Secondly, the framework is able to account for dynamic conditions, mainly due to its fast development cycles and possibility of customization, since it is entirely Python based. Regarding high complexity, in terms of larger and more complex

production systems, this study has not explored this prospect, nonetheless, the technologies behind the framework might be well suited for this, but would likely require further work and development of the framework. For these reasons, the framework could be expected to be a useful tool for human operators for quick evaluations and optimizations of smaller sub lines in a production system, with a limitation to the complexity of the production line of interest.

From an economical perspective, the framework is relatively cheap to run, as seen in Section 5.5, where a typical execution spent around 30 American cents. Typically, traditional simulation software can have a high license cost. In this sense, the framework can be considered a low cost alternative to professional simulation software.

In regards to the risk of long term deskilling of the workforce mentioned in Section 6.2, a company that uses the framework would have to take actions to prevent this. Since the framework might lead to loss of learning opportunities and expert tasks being taken of by AI, the company would need other strategies on how to maintain knowledge of simulation optimization in the workforce. These could for example include education, expectations of thorough declaration of all optimization decisions, or limiting the use of the framework in some way. Otherwise, if the employees were simply using the framework to replace manual work, it could aggravate knowledge depletion. Nonetheless, the human-in-the-loop aspect of the framework is especially important because it engages the human operator, which might reduce the risk of deskilling in the long term. However, as mentioned by [43] and [44], the deskilling due to AI mostly refers to loss of knowledge in the industry as a whole. Therefore, this problem can be difficult to address by a single company.

Chapter 7

Conclusions and Future Work

In this final chapter, the final conclusions of the thesis are presented. Furthermore, a section on possible avenues for future work follows. To reiterate, the research questions set out for this thesis were:

1. **RQ1:** How can a LLM-based, human-in-the-loop framework be used to help human operators optimize conflicting objectives in a production line?
2. **RQ2:** Can such a framework offer similar performance to traditional simulation model optimization?

7.1 Conclusions

To answer the first research question, a framework for simulation optimization was developed by continuing the work of [14], to allow a non-expert to find trade-off solutions in a production line. The framework is Python based, with which the user interacts through natural language in a terminal. The results indicate that a LLM-driven, human-in-the-loop framework built like the one in the study, has the possibility to optimize for conflicting objectives without requiring the user to have previous expert knowledge or skills regarding simulation based optimization nor coding. This is achieved through the use of prespecified scenarios for decision and objective variables. The frameworks ability to produce a bottleneck detection analysis can also allow the operator to gain deeper insight into the working of the production line in question.

Looking at the second research question, it is more concerned with the performance of such a framework. Based on the results and findings of this study a LLM-based framework seems to be able to achieve on par performance

with the more traditional approach of manually generating a simulation model, at least when comparing the resulting KPIs and quality of solutions sets. The framework developed in this study has however an increased risk of reliability issues compared to the manual approach, but these are also to a degree mitigated by the oversight and interaction of the user. However, the challenges remains that if the user is a non-expert, detecting structural and reasoning errors might be more difficult.

The performance and automation of the framework might also come at a cost. The use of LLMs expose users to the question of sustainability when using LLMs, as they require a lot of resources like energy and water. There is also the the potential long-term problem of the deskilling of the workforce which the users of such a framework must consider. All these potential problems and benefits must be weighed by a potential user to establish the usefulness of the framework for them.

7.2 Limitations

Even though the research questions were answered, there are some limiting factors to the study that might contradict the conclusions in the section above, which are described in the following paragraphs.

Firstly, not all features of the framework were examined in Chapter 5. Specifically, the frameworks ability to solve MOO problems with other decision variables than buffer capacities, and other objectives than WIP vs TP, and WIP vs SEC. Therefore, the full range of MOO solving capabilities of the framework have not been systematically tested, which weakens the arguments in the results.

Secondly, the number of experiments might be considered insufficient to get a full understanding of the performance and reliability of the framework. In Section 5.5, the framework was run 10 times, to analyze the most common errors produced and to evaluate its MOO solving abilities. Preferably, the framework would have been examined more extensively, to catch more potential errors and to make more confident conclusions. The first and second limitations, where mainly due to the time constraint and the scope of the thesis.

Thirdly, to make the results of this study more generalizable and relevant, a deeper study of different LLM models and a systematic test of hyper parameters would have deepened the understanding of how LLMs can be applied for this sort of task. For now, this study makes the assumption that the framework is compatible with many types of LLMs from different providers, but this was not empirically proved.

Finally, no non-experts was invited to try the framework, which weakens the argument that the framework would be suitable for non-experts to use. Having non-experts try the framework would have given insights into how to improve it and how it might be applied in a real world setting, to help non-experts solve manufacturing trade-offs.

7.3 Future work

One possible direction for future research is to test or expand the framework to more complex simulation models. This study was mostly concerned with applying a more complex optimization layer to the framework, while testing on a model of a production line (case A) that has been used earlier by [14] and [16]. To further explore the capabilities of the framework, testing on more complex and new, unseen, production lines would be necessary.

Another possibility for future research is to investigate the use of different LLMs in the framework, from different companies and different models. There can be potential gains from utilizing other models than those in this study. For example, a more complex model might yield performance gains that result in the framework being more reliable or generating more efficient code. There could also be instances where a less complicated model might yield similar performance but at a lower cost.

It might also be interesting to explore the possibility of giving LLM agents more of an independent role in the framework. For example, an orchestrator agent might be created that itself can decide how to optimize a particular production line by turning the different optimization methods into tools that are available for the agent to use. This could make it easier to automate the process as the orchestrator agent would only need to hold in its context window how to use the tools instead of risking to overload the context window by providing it with all possible ways to optimize the modeled production line.

Furthermore, the framework could be well suited for an extensive user study, in which non-experts are invited to try the framework to give their feedback and thoughts. As of now, there are no real scientific evidence that the framework would be easy to use or understand for a non-expert, which would negate the results of the study. The insights from such a study would yield much needed information on how to improve the framework and how it might be used in a real world setting.

There are more options for future research directions on this topic. However, the examples above are some that are closely related to the framework in this study.

References

- [1] N. Diaz-Elsayed, A. Jondral, S. Greinacher, D. Dornfeld, and G. Lanza, “Assessment of lean and green strategies by simulation of manufacturing systems in discrete production environments,” *CIRP Annals*, vol. 62, no. 1, pp. 475–478, Jan. 2013. doi: 10.1016/j.cirp.2013.03.066. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S000785061300067X>
- [2] L. Wang, A. H. C. Ng, and K. Deb, Eds., *Multi-objective Evolutionary Optimisation for Product Design and Manufacturing*. London: Springer, 2011. ISBN 978-0-85729-617-7 978-0-85729-652-8. [Online]. Available: <https://link.springer.com/10.1007/978-0-85729-652-8>
- [3] V. J. Mawson and B. R. Hughes, “The development of modelling tools to improve energy efficiency in manufacturing processes and systems,” *Journal of Manufacturing Systems*, vol. 51, pp. 95–105, Apr. 2019. doi: 10.1016/j.jmsy.2019.04.008. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0278612518304151>
- [4] M. Elbasheer, Y. Laili, F. Longo, V. Solina, Y. Tao, P. Veltri, Y. Zhang, and L. Zhang, “Natural language-driven production planning: integrating large language models with automatic simulation model generation in manufacturing systems,” *Journal of Intelligent Manufacturing*, Nov. 2025. doi: 10.1007/s10845-025-02732-z. [Online]. Available: <https://doi.org/10.1007/s10845-025-02732-z>
- [5] R. Skapinyecz, “Recent Trends in the Optimization of Logistics Systems Through Discrete-Event Simulation and Deep Learning,” *Algorithms*, vol. 18, no. 9, Sep. 2025. doi: 10.3390/a18090573. [Online]. Available: <https://www.mdpi.com/1999-4893/18/9/573>
- [6] V. Karkaria, Y.-K. Tsai, Y.-P. Chen, and W. Chen, “An optimization-centric review on integrating artificial intelligence and digital twin

- technologies in manufacturing,” *Engineering Optimization*, vol. 57, no. 1, pp. 161–207, Jan. 2025. doi: 10.1080/0305215X.2024.2434201. [Online]. Available: <https://doi.org/10.1080/0305215X.2024.2434201>
- [7] I. Jackson, M. Jesus Saenz, and D. Ivanov, “From natural language to simulations: applying AI to automate simulation modelling of logistics systems,” *International Journal of Production Research*, vol. 62, no. 4, pp. 1434–1457, Feb. 2024. doi: 10.1080/00207543.2023.2276811. [Online]. Available: <https://doi.org/10.1080/00207543.2023.2276811>
- [8] W. Liu, L. Chen, and Z. Tang, “Large Language Model Aided Multi-objective Evolutionary Algorithm: a Low-cost Adaptive Approach,” Oct. 2024, arXiv:2410.02301 [cs]. [Online]. Available: <http://arxiv.org/abs/2410.02301>
- [9] A. Tóth, L. Nagy, R. Kennedy, B. Bohuš, J. Abonyi, and T. Ruppert, “The human-centric Industry 5.0 collaboration architecture,” *MethodsX*, vol. 11, p. 102260, Dec. 2023. doi: 10.1016/j.mex.2023.102260. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2215016123002571>
- [10] X. Xu, Y. Lu, B. Vogel-Heuser, and L. Wang, “Industry 4.0 and Industry 5.0—Inception, conception and perception,” *Journal of Manufacturing Systems*, vol. 61, pp. 530–535, Oct. 2021. doi: 10.1016/j.jmsy.2021.10.006. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0278612521002119>
- [11] C. J. Turner and W. Garn, “Next generation DES simulation: A research agenda for human centric manufacturing systems,” *Journal of Industrial Information Integration*, vol. 28, p. 100354, Jul. 2022. doi: 10.1016/j.jii.2022.100354. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2452414X22000255>
- [12] R. Ojstersek, M. Brezocnik, and B. Buchmeister, “Multi-objective optimization of production scheduling with evolutionary computation: A review,” *International Journal of Industrial Engineering Computations*, vol. 11, Jan. 2020. doi: 10.5267/j.ijiec.2020.1.003
- [13] Y. Huang, S. Wu, W. Zhang, J. Wu, L. Feng, and K. C. Tan, “Autonomous Multi-Objective Optimization Using Large Language

- Model,” *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2025. doi: 10.1109/TEVC.2025.3561001. [Online]. Available: <https://ieeexplore.ieee.org/document/10965770>
- [14] N. Gegenmantel, *Automating the Path from Data to Decision: Generating and Adapting Simulations with Gen-AI*, 2025. [Online]. Available: <https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-560614>
- [15] OpenAI, “API-plattform,” accessed March 2026. [Online]. Available: <https://openai.com/sv-SE/api/>
- [16] T. Schmitt, N. Gegenmantel, M. Urenda Moris, P. Mårtensson, and K. Amouzgar, “LLM-driven discrete-event simulation: A generative AI framework for automated model generation, adaptation, and evaluation in manufacturing,” *Journal of Manufacturing Systems*, vol. 85, pp. 642–661, Apr. 2026. doi: 10.1016/j.jmsy.2026.02.015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0278612526000427>
- [17] C. A. Escobar, M. E. McGovern, and R. Morales-Menendez, “Quality 4.0: a review of big data challenges in manufacturing,” *Journal of Intelligent Manufacturing*, vol. 32, no. 8, pp. 2319–2334, Dec. 2021. doi: 10.1007/s10845-021-01765-4. [Online]. Available: <https://doi.org/10.1007/s10845-021-01765-4>
- [18] D. Pietsch, M. Matthes, U. Wieland, S. Ihlenfeldt, and T. Munkelt, “Root Cause Analysis in Industrial Manufacturing: A Scoping Review of Current Research, Challenges and the Promises of AI-Driven Approaches,” *Journal of Manufacturing and Materials Processing*, vol. 8, no. 6, p. 277, Dec. 2024. doi: 10.3390/jmmp8060277. [Online]. Available: <https://www.mdpi.com/2504-4494/8/6/277>
- [19] S. S. Sengar, A. B. Hasan, S. Kumar, and F. Carroll, “Generative artificial intelligence: a systematic review and applications,” *Multimedia Tools and Applications*, vol. 84, no. 21, pp. 23 661–23 700, Jun. 2025. doi: 10.1007/s11042-024-20016-1. [Online]. Available: <https://doi.org/10.1007/s11042-024-20016-1>
- [20] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, “A Comprehensive Overview of Large Language Models,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, pp. 106:1–106:72, Aug. 2025. doi: 10.1145/3744746. [Online]. Available: <https://dl.acm.org/doi/10.1145/3744746>

- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” Aug. 2023, arXiv:1706.03762 [cs]. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [22] X. Tang, H. Chen, D. Lin, and K. Li, “Harnessing LLMs for multi-dimensional writing assessment: Reliability and alignment with human judgments,” *Heliyon*, vol. 10, no. 14, p. e34262, Jul. 2024. doi: 10.1016/j.heliyon.2024.e34262. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405844024102939>
- [23] C. Arora, A. I. Sayeed, S. Licorish, F. Wang, and C. Treude, “Optimizing Large Language Model Hyperparameters for Code Generation,” Aug. 2024, arXiv:2408.10577 [cs]. [Online]. Available: <http://arxiv.org/abs/2408.10577>
- [24] J. Wiesinger, P. Marlow, and V. Vuskovic, “Agents,” Tech. Rep., 2025. [Online]. Available: <https://www.kaggle.com/whitepaper-agents>
- [25] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The Rise and Potential of Large Language Model Based Agents: A Survey,” Sep. 2023, arXiv:2309.07864 [cs]. [Online]. Available: <http://arxiv.org/abs/2309.07864>
- [26] M. Choudhary, B. Agarwal, and V. Goyal, “Hate Speech Detection: Leveraging LLM-GPT2 with Fine-Tuning and Multi-Shot Techniques,” *Procedia Computer Science*, vol. 258, pp. 2817–2825, Jan. 2025. doi: 10.1016/j.procs.2025.04.542. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050925016461>
- [27] D. Han, A. Raglin, and D. Summers-Stay, “Negative Nudging to Quantify the LLM Hallucination,” in *Artificial Intelligence in HCI*, H. Degen and S. Ntoa, Eds. Cham: Springer Nature Switzerland, 2025. doi: 10.1007/978-3-031-93418-6_11pp.152 – –167.
- [28] K. G. Barman, N. Wood, and P. Pawlowski, “Beyond transparency and explainability: on the need for adequate and contextualized user guidelines for LLM use,” *Ethics and Information Technology*, vol. 26, no. 3,

- p. 47, Jul. 2024. doi: 10.1007/s10676-024-09778-2. [Online]. Available: <https://doi.org/10.1007/s10676-024-09778-2>
- [29] C. Wang, Z. Li, C. Gao, W. Wang, T. Peng, H. Huang, Y. Deng, S. Wang, and M. Lyu, “Exploring Multi-Lingual Bias of Large Code Models in Code Generation,” *ACM Trans. Softw. Eng. Methodol.*, Jan. 2026. doi: 10.1145/3786793 Just Accepted. [Online]. Available: <https://dl.acm.org/doi/10.1145/3786793>
- [30] O. Liu, D. Fu, D. Yogatama, and W. Neiswanger, “DeLLMa: Decision Making Under Uncertainty with Large Language Models,” Oct. 2024, arXiv:2402.02392 [cs]. [Online]. Available: <http://arxiv.org/abs/2402.02392>
- [31] A. Greasley, *Simulating Business Processes for Descriptive, Predictive, and Prescriptive Analytics*. De Gruyter, Oct. 2019. [Online]. Available: <https://www.degruyterbrill.com/document/doi/10.1515/9781547400690/html>
- [32] H. Wang, M. Olhofer, and Y. Jin, “A mini-review on preference modeling and articulation in multi-objective optimization: current status and challenges,” *Complex & Intelligent Systems*, vol. 3, no. 4, pp. 233–245, Dec. 2017. doi: 10.1007/s40747-017-0053-9. [Online]. Available: <https://doi.org/10.1007/s40747-017-0053-9>
- [33] M. Ehrgott, M. Köksalan, M. Kadziński, and K. Deb, “Fifty years of multi-objective optimization and decision-making: From mathematical programming to evolutionary computation,” *European Journal of Operational Research*, vol. 330, no. 1, pp. 1–25, Apr. 2026. doi: 10.1016/j.ejor.2025.06.012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0377221725004849>
- [34] L. Pan, W. Xu, L. Li, C. He, and R. Cheng, “Adaptive simulated binary crossover for rotated multi-objective optimization,” *Swarm and Evolutionary Computation*, vol. 60, p. 100759, Feb. 2021. doi: 10.1016/j.swevo.2020.100759. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2210650220304120>
- [35] H. Jain and K. Deb, “Parent to Mean-Centric Self-Adaptation in SBX Operator for Real-Parameter Optimization,” in *Swarm, Evolutionary, and Memetic Computing*, B. K. Panigrahi, P. N. Suganthan, S. Das, and S. C. Satapathy, Eds. Berlin, Heidelberg: Springer, 2011. doi: 10.1007/978-3-642-27172-4_37pp.299 – –306.

- [36] J. L. Carles-Bou and S. F. Galán, “Self-adaptive polynomial mutation in NSGA-II,” *Soft Computing*, vol. 27, no. 23, pp. 17 711–17 727, Dec. 2023. doi: 10.1007/s00500-023-09049-0. [Online]. Available: <https://doi.org/10.1007/s00500-023-09049-0>
- [37] I. Giagkiozis and P. J. Fleming, “Methods for multi-objective optimization: An analysis,” *Information Sciences*, vol. 293, pp. 338–350, Feb. 2015. doi: 10.1016/j.ins.2014.08.071. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025514009074>
- [38] Y. Zhang, R. Cheng, G. Yi, and K. C. Tan, “A Systematic Survey on Large Language Models for Evolutionary Optimization: From Modeling to Solving,” Jan. 2026, arXiv:2509.08269 [cs]. [Online]. Available: <http://arxiv.org/abs/2509.08269>
- [39] M. Li and X. Yao, “Quality Evaluation of Solution Sets in Multiobjective Optimisation: A Survey,” *ACM Comput. Surv.*, vol. 52, no. 2, pp. 26:1–26:38, Mar. 2019. doi: 10.1145/3300148. [Online]. Available: <https://dl.acm.org/doi/10.1145/3300148>
- [40] K. Deb, K. Miettinen, and D. Sharma, “A Hybrid Integrated Multi-Objective Optimization Procedure for Estimating Nadir Point,” in *Evolutionary Multi-Criterion Optimization*, M. Ehrgott, C. M. Fonseca, X. Gandibleux, J.-K. Hao, and M. Sevaux, Eds. Berlin, Heidelberg: Springer, 2009. doi: 10.1007/978-3-642-01020-0_44pp.569 – –583.
- [41] A. Ingemansson, T. Ylipää, and G. S. Bolmsjö, “Reducing bottle-necks in a manufacturing system with automatic data collection and discrete-event simulation,” *Journal of Manufacturing Technology Management*, vol. 16, no. 6, pp. 615–628, Sep. 2005. doi: 10.1108/17410380510609474. [Online]. Available: <https://doi.org/10.1108/17410380510609474>
- [42] J. Bernedixen, L. Pehrsson, A. H. Ng, and T. Antonsson, “Simulation-based multi-objective bottleneck improvement: Towards an automated toolset for industry,” in *2015 Winter Simulation Conference (WSC)*, Dec. 2015. doi: 10.1109/WSC.2015.7408331. ISSN 1558-4305 pp. 2183–2194, iSSN: 1558-4305. [Online]. Available: <https://ieeexplore.ieee.org/document/7408331>
- [43] J. Krook, “When autonomy breaks: the hidden existential risk of AI,” *AI & SOCIETY*, vol. 40, no. 8, pp. 6011–6024, Dec. 2025. doi: 10.1007/s00146-025-02397-5. [Online]. Available: <https://doi.org/10.1007/s00146-025-02397-5>

- [44] A. Ardichvili, “The Impact of Artificial Intelligence on Expertise Development: Implications for HRD,” *Advances in Developing Human Resources*, vol. 24, no. 2, pp. 78–98, May 2022. doi: 10.1177/15234223221077304. [Online]. Available: <https://doi.org/10.1177/15234223221077304>
- [45] A. Domínguez Hernández, A. M. Perini, S. Hadjiloizou, A. Borda, S. Mahomed, and D. Leslie, “Towards a sociotechnical ecology of artificial intelligence: power, accountability, and governance in a global context,” *AI and Ethics*, vol. 6, no. 1, p. 43, Dec. 2025. doi: 10.1007/s43681-025-00902-6. [Online]. Available: <https://doi.org/10.1007/s43681-025-00902-6>
- [46] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, “LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254–2268, Sep. 2024. doi: 10.1109/TSE.2024.3428972. [Online]. Available: <https://ieeexplore.ieee.org/document/10606356>
- [47] B. Wan, Z. Wei, S. Wang, J. Huang, and C. Hu, “Structure-guided function-level code generation with LLMs via UML activity diagrams,” *Neurocomputing*, vol. 669, p. 132502, Mar. 2026. doi: 10.1016/j.neucom.2025.132502. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231225031741>
- [48] M. Gogolla, C. Kobryn, G. Goos, J. Hartmanis, and J. Van Leeuwen, Eds., *UML 2001 — The Unified Modeling Language. Modeling Languages, Concepts, and Tools: 4th International Conference Toronto, Canada, October 1–5, 2001 Proceedings*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2001, vol. 2185. [Online]. Available: <http://link.springer.com/10.1007/3-540-45441-1>
- [49] F. Liu, X. Lin, Z. Wang, S. Yao, X. Tong, M. Yuan, and Q. Zhang, “Large Language Model for Multi-objective Evolutionary Optimization,” Mar. 2024, arXiv:2310.12541 [cs]. [Online]. Available: <http://arxiv.org/abs/2310.12541>
- [50] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large Language Models as Optimizers,” Apr. 2024, arXiv:2309.03409 [cs]. [Online]. Available: <http://arxiv.org/abs/2309.03409>
- [51] Y. Cheng, C. Zhang, Z. Zhang, X. Meng, S. Hong, W. Li, Z. Wang, Z. Wang, F. Yin, J. Zhao, and X. He, “Exploring Large Language Model

- based Intelligent Agents: Definitions, Methods, and Prospects,” Jan. 2024, arXiv:2401.03428 [cs]. [Online]. Available: <http://arxiv.org/abs/2401.03428>
- [52] X. Wu, Y. Zhong, J. Wu, B. Jiang, and K. C. Tan, “Large Language Model-Enhanced Algorithm Selection: Towards Comprehensive Algorithm Representation,” May 2024, arXiv:2311.13184 [cs]. [Online]. Available: <http://arxiv.org/abs/2311.13184>
- [53] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang, “Multi-objective Evolution of Heuristic Using Large Language Model,” Feb. 2025, arXiv:2409.16867 [cs]. [Online]. Available: <http://arxiv.org/abs/2409.16867>
- [54] F. Assad, J. Patsavellas, and K. Salonitis, “Enhancing sustainability in manufacturing through cognitive digital twins powered by generative artificial intelligence,” *Procedia CIRP*, vol. 130, pp. 677–682, Jan. 2024. doi: 10.1016/j.procir.2024.10.147. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2212827124013040>
- [55] D. Polojärvi, E. Palmer, and C. Dunford, “A systematic literature review of sociotechnical systems in systems engineering,” *Systems Engineering*, vol. 26, no. 4, pp. 482–504, 2023. doi: 10.1002/sys.21664. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/sys.21664>
- [56] L. K. Soiferman, “Compare and Contrast Inductive and Deductive Research Approaches,” Tech. Rep., Apr. 2010, publication Title: Online Submission ERIC Number: ED542066. [Online]. Available: <https://eric.ed.gov/?id=ED542066>
- [57] Y. S. Park, L. Konge, and A. R. Artino, Jr, “The Positivism Paradigm of Research,” *Academic Medicine*, vol. 95, no. 5, pp. 690–694, May 2020. doi: 10.1097/ACM.0000000000003093. [Online]. Available: <https://doi.org/10.1097/ACM.0000000000003093>
- [58] Wikipedia, “Comma-separated values,” May 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Comma-separated_values&oldid=1355452104
- [59] Visual Studio Code, “Visual Studio Code - The open source AI code editor,” accessed March 2026. [Online]. Available: <https://code.visualstudio.com/>
- [60] SimPy, “SimPy - Discrete event simulation for Python,” accessed March 2026. [Online]. Available: <https://simpy.readthedocs.io/en/latest/>

- [61] J. Blank and K. Deb, “Pymoo: Multi-Objective Optimization in Python,” *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020. doi: 10.1109/ACCESS.2020.2990567. [Online]. Available: <https://ieeexplore.ieee.org/document/9078759/>
- [62] Mermaid, “Mermaid,” accessed April 2026. [Online]. Available: <https://mermaid.js.org>
- [63] Wikipedia, “PNG,” Jun. 2024. [Online]. Available: <https://sv.wikipedia.org/w/index.php?title=PNG&oldid=55101746>
- [64] Evoma, “FACTS Analyzer documentation,” accessed March 2026. [Online]. Available: <https://software.evoma.se/facts/help/latest/>
- [65] No Author, “Introducing JSON,” accessed May 2026. [Online]. Available: <https://www.json.org/json-en.html>